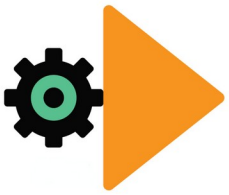

E-TOOLS AI CORPORATION

Abe Pro

Programming Manual

The complete programming guide for the Abe Pro Compiler (`abec`) — a statically-typed, ahead-of-time compiled language that lowers to LLVM IR and native code, with a batteries-included runtime.

Part I: Level 1 — General Application Programming
Part II: Level 2 — Machine Learning
Appendix: Runtime Function Reference (523 functions)

Version 1.0.0 · July 2026

Contents

PART I

Level 1: General Application Programming

1. Getting Started

1.1 What is Abe?

Abe is a typed, memory-safe systems language that compiles `.abe` source to native code through LLVM. `abec` is the compiler. Programs are ordinary native executables — no virtual machine, no runtime interpreter — and link a small runtime library that provides strings, collections, JSON, crypto, HTTP, and more (Part III).

1.2 Installing `abec`

Download the prebuilt Linux x86-64 compiler and the runtime it links. Keep them side by side — `abec` finds an adjacent `runtime/` automatically:

```
## The compiler (a single self-contained binary):
curl -fsSL -o abec \
  https://github.com/E-Tools-AI-Corporation/abec/releases/latest/download/abec-
linux-x86_64
chmod +x abec
./abec --version          # -> abec 1.0.0 (v4.1 grammar)

## The runtime (needed to link native programs):
curl -fsSL -O \
  https://github.com/E-Tools-AI-Corporation/abec/releases/latest/download/abe-
runtime-1.0.0-linux-x86_64.tar.gz
tar xzf abe-runtime-1.0.0-linux-x86_64.tar.gz   # creates ./runtime next to abec
```

Building a native executable runs the LLVM toolchain, so **`clang` and `llc` must be on your `PATH`** (install your distribution's `clang` / `llvm` packages). `abec` runs free for a **30-day evaluation**; after that it needs a license — see Chapter 14.

> If `abec` and `runtime/` live in different places, point the compiler at the runtime with `export ABE_RUNTIME_DIR=/path/to/runtime`.

1.3 Your first program

Create `hello.abe`:

```
function main(): i64 {
    console.log("Hello, Abe!");
    return 0;
}
```

Compile and run it:

```
./abec hello.abe -o hello     # compile to a native binary
./hello                       # prints: Hello, Abe!
```

1.4 The execution model — `main()` is required

Every Abe program starts at `main`, which **must** be declared as `function main(): i64`. Its return value is the process exit code (`0` means success). Unlike scripting languages, top-level statements do **not** execute on their own — put your logic inside `main` (or functions it calls).

1.5 The `abec` toolchain

Command	What it does
`abec FILE -o OUT`	Compile `FILE` to the native executable `OUT`
`abec --check FILE`	Parse + type-check only (no code generated)
`abec --emit-ll FILE`	Print the generated LLVM IR to stdout
`abec --version`	Print the compiler version

`console.log(...)` prints its arguments separated by spaces, followed by a newline. It accepts strings, numbers, and booleans.

---

2. Types and Variables

2.1 Declaring variables

Abe has three binding forms. `let` and `var` are mutable; `const` is immutable. A type annotation (`: i64`) is optional when the type can be inferred from the initializer.

```
function main(): i64 {
    let count: i64 = 3;       // mutable, annotated
    const name = "Abe";       // immutable, inferred
    var total = count + 1;    // mutable, inferred
    if (total != 4) return 1;
    if (name != "Abe") return 2;
    return 0;
}
```

> Variables also work at **module scope** (top level): a `const` for shared configuration, or a `let` for module-wide mutable state that functions read and update. See §2.7.

2.2 Numeric types

The core numeric types are `i64` (64-bit signed integer) and `f64` (64-bit float). Arithmetic uses the usual precedence — `*`, `/`, `%` bind tighter than `+`, `-`.

```
function main(): i64 {
    let a: i64 = 3 + 4 * 2;     // 11 — '*' binds tighter than '+'
    let rem: i64 = 10 % 3;      // 1 — remainder
    let pow: i64 = 2 ** 10;     // 1024 — exponent operator
    console.log(a, rem, pow);   // prints: 11 1 1024
    return 0;
}
```

The `**` operator also works on floats (e.g. `9.0 ** 0.5` is `3.0`).

Bitwise and shift operators work on integers:

```
function main(): i64 {
    if ((6 & 3) != 2)   return 1;   // AND
    if ((6 | 1) != 7)   return 2;   // OR
    if ((6 ^ 3) != 5)   return 3;   // XOR
    if ((1 << 4) != 16) return 4;   // shift left
    if ((32 >> 2) != 8) return 5;   // shift right
    return 0;
}
```

Floating-point values use `f64`:

```
function main(): i64 {
    let price: f64 = 2.5;
    let total: f64 = price * 4.0;
    if (total != 10.0) return 1;
    return 0;
}
```

2.3 Strings and template literals

Strings concatenate with `+`. Template literals (backticks) interpolate expressions with `${...}`:

```
function main(): i64 {
    let who: string = "Abe";
    let version: i64 = 4;
    let greeting = `Hello, ${who} — v${version}!`;  // "Hello, Abe — v4!"
    console.log(greeting);
    let joined = "a" + "b" + "c";
    if (joined != "abc") return 1;
    return 0;
}
```

2.4 Arrays

Array literals are written with `[...]`. Read the length with `.length`, index with `[i]`, and iterate with `for ... of`:

```
function main(): i64 {
    let nums = [10, 20, 30];
    if (nums.length != 3) return 1;
    if (nums[1] != 20) return 2;
    let sum = 0;
    for (const n of nums) { sum = sum + n; }
    if (sum != 60) return 3;
    return 0;
}
```

Arrays of numbers, strings, and booleans all work; functional methods (`.map`, `.filter`, `.forEach`) are covered in the Collections chapter.

2.5 Objects

Object literals map string keys to values and are read with `.field`:

```
function main(): i64 {
    let user = { name: "Alice", role: "admin" };
    if (user.name != "Alice") return 1;
    if (user.role != "admin") return 2;
    return 0;
}
```

> In this release object-literal members hold **string** values (the form used by JSON and configuration). Numeric-valued members are a documented limitation and are slated for a future release.

2.6 Null safety

The nullish-coalescing operator `??` yields its left side unless that side is null, in which case it yields the right side. The ternary operator `?:` chooses between two values:

```
function pick(value: string): string {
    return value ?? "default";          // nullish coalescing
}
function main(): i64 {
    let label = 7 > 3 ? "big" : "small";  // ternary
    if (label != "big") return 1;
    if (pick("set") != "set") return 2;
    return 0;
}
```

2.7 Module-scope variables

Declarations at the top level of a file (outside any function) are visible to every function in the module: a `const` for shared configuration, or a `let` for module-wide mutable state.

```
const MAX_RETRIES: i64 = 3;     // module-level const
let requestCount: i64 = 0;      // module-level mutable state

function record(): void { requestCount = requestCount + 1; }

function main(): i64 {
    record();
    record();
    if (MAX_RETRIES != 3) return 1;
    if (requestCount != 2) return 2;
    return 0;
}
```

> Functions may be declared in any order relative to the module-level variables they use — initialization runs before `main`.

---

3. Functions

3.1 Declaring functions

A function declares typed parameters and a return type. The body returns a value of that type (or nothing, for `void`).

```
function add(a: i64, b: i64): i64 {   // typed params + return type
    return a + b;
}
function greet(name: string): string {
    return "hi " + name;
}
function main(): i64 {
    if (add(2, 3) != 5) return 1;
    if (greet("Abe") != "hi Abe") return 2;
    return 0;
}
```

A `void` function returns no value — it runs for its side effects:

```
function announce(msg: string): void {  // void: returns no value
    console.log(msg);
}
function main(): i64 {
    announce("ready");   // prints: ready
    return 0;
}
```

3.2 Default parameters

A parameter may declare a default value, used when the caller omits that argument. (Defaults apply from the trailing end of the parameter list.)

```
function greet(name: string = "world"): string {
    return "hi " + name;
}
function main(): i64 {
    if (greet() != "hi world") return 1;    // default used
    if (greet("Abe") != "hi Abe") return 2;  // overridden
    return 0;
}
```

3.3 Recursion

Functions may call themselves:

```
function fib(n: i64): i64 {
    if (n < 2) return n;
```

```
    return fib(n - 1) + fib(n - 2);
}
function main(): i64 {
    if (fib(10) != 55) return 1;
    return 0;
}
```

3.4 Arrow functions and first-class function values

An **arrow function** `(params) => expr` is a function value. It can be stored in a variable and called, passed as an argument, or returned:

```
function main(): i64 {
    let inc = (x: i64): i64 => x + 1;
    let add = (a: i64, b: i64): i64 => a + b;
    if (inc(9) != 10) return 1;
    if (add(20, 22) != 42) return 2;
    return 0;
}
```

Arrow functions are also the natural argument to the collection methods `.map`, `.filter`, and `.forEach` (see the Collections chapter).

3.5 Higher-order functions

A function can take another function as a parameter, typed with a **function type** `(name: T) => U`. Both named functions and arrows can be passed:

```
function apply(f: (n: i64) => i64, x: i64): i64 {
    return f(x);
}
function double(n: i64): i64 { return n * 2; }
function main(): i64 {
    if (apply(double, 21) != 42) return 1;                  // pass a named function
    if (apply((n: i64): i64 => n + 1, 9) != 10) return 2;   // pass an arrow
    return 0;
}
```

> Function-type annotations require **named** parameters — write `(n: i64) => i64`, not `(i64) => i64`.

3.6 Generic functions

A function can be parameterized by a type with `<T>`. Generics are **pass-through**: a `T` value is carried unchanged from input to output. This covers identity and forwarding helpers:

```
function identity<T>(x: T): T { return x; }
function main(): i64 {
    let n: i64 = identity(5);
    let s: string = identity("abe");
    if (n != 5) return 1;
    if (s != "abe") return 2;
    return 0;
```

```
}
```

The concrete type is inferred from the argument. Annotate the result (`let n: i64 = …`) so the value is recovered at its real type.

> **Limit:** a `T` value is opaque — you can pass it through, store it, and return it, but you cannot do arithmetic or comparison on it inside the generic function. Explicit call-site type arguments (`identity<i64>(5)`) are not yet supported; rely on inference.

3.7 Async functions

`async` functions and `await` are accepted, but the runtime is **single-threaded and runs them synchronously** — `await e` simply evaluates `e`. There is no event loop.

```
async function compute(x: i64): i64 {
    return x * 2;
}
function main(): i64 {
    let r = await compute(21);   // runs synchronously
    if (r != 42) return 1;
    return 0;
}
```

> For real concurrency, use **actors** and **task groups** (a separate topic), which provide `Sendable`-checked message passing rather than an async event loop.

---

4. Control Flow

4.1 If / else

```
function classify(n: i64): string {
    if (n < 0) {
        return "negative";
    } else if (n == 0) {
        return "zero";
    } else {
        return "positive";
    }
}
```

4.2 Loops

A `while` loop runs while its condition holds:

```
function main(): i64 {
    let i = 0;
    let sum = 0;
    while (i < 5) {
        sum = sum + i;
```

```
        i = i + 1;
    }
    if (sum != 10) return 1;   // 0+1+2+3+4
    return 0;
}
```

A `do … while` loop runs the body at least once before testing:

```
let n = 0;
do {
    n = n + 1;
} while (n < 3);
```

A C-style `for` loop has init, condition, and step clauses:

```
let product = 1;
for (let i = 1; i <= 5; i = i + 1) {
    product = product * i;   // 5! = 120
}
```

`for … of` iterates the elements of an array (numbers, strings, or booleans):

```
function main(): i64 {
    let total = 0;
    for (const n of [10, 20, 30]) { total = total + n; }   // 60
    let joined = "";
    for (const word of ["a", "b", "c"]) { joined = joined + word; }  // "abc"
    if (total != 60 || joined != "abc") return 1;
    return 0;
}
```

> Use `for … of` to iterate values. (`for … in` over indices is not yet lowered — index with a C-style `for` loop when you need positions.)

4.3 break and continue

`break` exits the nearest loop; `continue` skips to its next iteration:

```
let sum = 0;
for (let i = 0; i < 10; i = i + 1) {
    if (i == 5) break;          // stop the loop
    if (i % 2 == 0) continue;   // skip even numbers
    sum = sum + i;              // 1 + 3 = 4
}
```

4.4 switch

`switch` compares a value against `case` labels (integers or strings). A case runs until `break`; cases without a `break` **fall through** to the next, so stacking case labels shares one body. `default` runs when nothing matches.

```
function describe(n: i64): string {
    let label = "";
    switch (n) {
        case 1:
            label = "one";
            break;
        case 2:
        case 3:
            label = "few";      // 2 and 3 share this body
            break;
        default:
            label = "many";
    }
    return label;
}
```

4.5 Pattern matching with `match`

`match` is an **expression** — it evaluates to a value. Each arm is `pattern => result`, tried top to bottom. Patterns include literals, ranges (`lo ... hi`), or-patterns (`a | b`), a binding identifier, a wildcard `_`, and an optional `if` guard:

```
function grade(score: i64): string {
    return match score {
        90 ... 100 => "A",          // range
        80 ... 89  => "B",
        0          => "F",          // literal
        s if s < 0 => "invalid",    // binding + guard
        _          => "C"           // wildcard (catch-all)
    };
}
```

`match` also **destructures** arrays and objects, binding parts to names:

```
function main(): i64 {
    let point = [3, 4];
    let r = match point {
        [x, y] => x + y,    // bind elements; also `[first, ...rest]`
        _ => 0
    };
    if (r != 7) return 1;

    let user = { role: "admin" };
    let access = match user {
        { role } => role,   // bind the `role` field
        _ => "none"
    };
    if (access != "admin") return 2;
    return 0;
}
```

> Constructor patterns (`Some(x)`) are not yet available — they depend on `enum`/tagged-union support landing in a future release.

---

5. Collections

5.1 Arrays

Arrays are ordered, growable lists. Create them with `[...]`, read `.length`, index with `[i]`, append with `.push`, and remove the last element with `.pop`:

```
function main(): i64 {
    let xs = [10, 20, 30];
    if (xs.length != 3) return 1;
    if (xs[0] != 10) return 2;
    xs.push(40);            // append -> [10, 20, 30, 40]
    if (xs.length != 4) return 3;
    if (xs[3] != 40) return 4;
    xs.pop();               // remove the last element -> [10, 20, 30]
    if (xs.length != 3) return 5;
    return 0;
}
```

`.pop` also *returns* the element it removed, so you can capture it:

```
function main(): i64 {
    let xs = [10, 20, 30];
    let last = xs.pop();    // last == 30, xs == [10, 20]
    if (last != 30) return 1;
    if (xs.length != 2) return 2;
    return 0;
}
```

Iterate with `for … of` (see Chapter 4). Arrays of numbers, strings, and booleans are all supported.

5.2 map

`.map` builds a new array by transforming each element with a callback. Its result is itself an array you can index or iterate:

```
function main(): i64 {
    let doubled = [1, 2, 3].map(x => x * 2);   // [2, 4, 6]
    if (doubled.length != 3) return 1;
    if (doubled[2] != 6) return 2;
    return 0;
}
```

5.3 filter

`.filter` keeps the elements for which the callback returns true:

```
function main(): i64 {
    let evens = [1, 2, 3, 4, 5, 6].filter(x => x % 2 == 0);  // [2, 4, 6]
    if (evens.length != 3) return 1;
    let sum = 0;
    for (const n of evens) { sum = sum + n; }   // 12
    if (sum != 12) return 2;
    return 0;
}
```

5.4 forEach

`.forEach` runs a callback for each element, for its side effects. The callback can update variables in the enclosing scope:

```
function main(): i64 {
    let total = 0;
    [10, 20, 30].forEach(n => { total = total + n; });
    if (total != 60) return 1;
    return 0;
}
```

5.5 Chaining

Because `.map` and `.filter` return arrays, they chain:

```
let result = [1, 2, 3, 4, 5, 6]
    .filter(x => x > 2)     // [3, 4, 5, 6]
    .map(x => x * 10);      // [30, 40, 50, 60]
```

5.6 reduce

`.reduce(callback, initial)` threads an accumulator through the array: the callback receives `(accumulator, element)` and returns the next accumulator. The accumulator's type is that of the `initial` value, and may differ from the element type:

```
function main(): i64 {
    let xs = [1, 2, 3, 4];
    let sum = xs.reduce((acc, x) => acc + x, 0);        // 10
    if (sum != 10) return 1;
    let product = xs.reduce((acc, x) => acc * x, 1);    // 24
    if (product != 24) return 2;
    // Accumulator type differs from the element type:
    let totalLen = ["a", "bb", "ccc"].reduce((acc, w) => acc + w.length, 0);
    if (totalLen != 6) return 3;
    let joined = ["a", "b", "c"].reduce((acc, x) => acc + x, "");  // "abc"
    if (joined != "abc") return 4;
    return 0;
}
```

5.7 find

`.find(predicate)` returns the first element for which the predicate is true, or `null` when none match. Compare the result against `null` before using it:

```
function main(): i64 {
    let users = [{ name: "ada" }, { name: "alan" }, { name: "grace" }];
    let u = users.find(r => r.name == "alan");
    if (u == null) return 1;
    if (u.name != "alan") return 2;
    let missing = users.find(r => r.name == "turing");
    if (missing != null) return 3;       // null: no match
    return 0;
}
```

`.find` is for arrays of objects or strings, where `null` is a sound "not found". For arrays of numbers or booleans — where every value is a valid element and there is no spare "not found" value — use `.indexOf` (which returns `-1` when absent) or `.filter` instead.

5.8 Slicing

`.slice(start, end)` returns a new array with the elements from `start` up to (but not including) `end`. The original array is left unchanged, and the returned elements keep their type — numbers read back as numbers:

```
function main(): i64 {
    let xs = [10, 20, 30, 40, 50];
    let mid = xs.slice(1, 4);    // [20, 30, 40]
    if (mid.length != 3) return 1;
    if (mid[0] != 20) return 2;
    let sum = 0;
    for (const n of mid) { sum = sum + n; }   // 90
    if (sum != 90) return 3;
    return 0;
}
```

5.9 Membership

`.includes(v)` reports whether a value is present, and `.indexOf(v)` returns the index of the first match (or `-1`). Both compare **by value**, and work for arrays of numbers and of strings:

```
function main(): i64 {
    let nums = [10, 20, 30];
    if (!nums.includes(20)) return 1;
    if (nums.indexOf(30) != 2) return 2;
    if (nums.indexOf(99) != -1) return 3;

    let names = ["ada", "alan", "grace"];
    if (!names.includes("alan")) return 4;
    if (names.indexOf("turing") != -1) return 5;
    return 0;
```

```
}
```

5.10 Strings ↔ arrays

`.split` turns a string into an array of pieces; `.join` turns an array of strings back into one string. Combined with `.map`, this covers most text processing:

```
function main(): i64 {
    let parts = "a,b,c".split(",");          // ["a", "b", "c"]
    if (parts.length != 3) return 1;
    let trimmed = " x , y ".split(",")
        .map(p => p.trim())
        .join("|");                          // "x|y"
    if (trimmed != "x|y") return 2;
    return 0;
}
```

---

6. Error Handling

6.1 throw and try / catch

Raise an error with `throw`, and handle it with `try` / `catch`. The thrown value is bound to the `catch` parameter:

```
function main(): i64 {
    try {
        throw "boom";        // raise an error
        return 1;            // unreachable
    } catch (e) {
        return 0;            // handled
    }
}
```

The caught value is usable — most often a string message:

```
function main(): i64 {
    try {
        throw "access denied";
    } catch (e) {
        console.log(e);      // prints: access denied
    }
    return 0;
}
```

You can throw any value — a string, an object (`throw { code: "E1" }`), etc.

6.2 finally

A `finally` block runs whether or not an error was thrown — use it for cleanup that must always happen:

```
function cleanupCount(doThrow: bool): i64 {
    let cleanups = 0;
    try {
        if (doThrow) { throw "x"; }
    } catch (e) {
        // swallow
    } finally {
        cleanups = cleanups + 1;   // runs on BOTH paths
    }
    return cleanups;               // always 1
}
```

6.3 Propagation

A `throw` propagates up the call stack until a `try`/`catch` handles it — so a deep helper can fail and a caller can recover, without threading error codes through every return:

```
function parseAmount(s: string): i64 {
    if (s == "") { throw "empty input"; }
    return 42;
}
function main(): i64 {
    try {
        let n = parseAmount("");   // throws inside the callee
        return 1;                  // unreachable
    } catch (e) {
        return 0;                  // caught here
    }
}
```

> **Style:** for *expected* outcomes (a lookup miss, optional input), prefer returning a value (e.g. `??` with a default, or a sentinel) over `throw`. Reserve `throw` for genuinely exceptional conditions.

---

7. Classes and Enums

7.1 Classes

A class groups typed fields with a `constructor` and methods. Create an instance with `new`; inside the class, `this` refers to the instance:

```
class Point {
    x: i64;
    y: i64;
    constructor(x: i64, y: i64) {
        this.x = x;
        this.y = y;
    }
    sum(): i64 {
        return this.x + this.y;
```

```
    }
}
function main(): i64 {
    let p = new Point(2, 3);
    if (p.x != 2) return 1;        // field access
    if (p.sum() != 5) return 2;    // method call
    return 0;
}
```

Methods can call other methods on `this`, and `void` methods mutate fields:

```
class Counter {
    n: i64;
    constructor() { this.n = 0; }
    bump(): void { this.n = this.n + 1; }
    twice(): void { this.bump(); this.bump(); }
    value(): i64 { return this.n; }
}
```

7.2 Inheritance

A class may `extends` one base class. The derived class **inherits** the base's fields and methods and can **override** methods:

```
class Animal {
    speak(): string { return "..."; }
    kind(): string { return "animal"; }
}
class Dog extends Animal {
    speak(): string { return "woof"; }   // override
}
function main(): i64 {
    let d = new Dog();
    if (d.speak() != "woof")   return 1;   // overridden
    if (d.kind()  != "animal") return 2;   // inherited
    return 0;
}
```

Inherited fields and constructors work too — a derived class with no constructor of its own uses the base's:

```
class Animal {
    name: string;
    constructor(n: string) { this.name = n; }
    describe(): string { return this.name; }
}
class Dog extends Animal {
    bark(): string { return this.name + " barks"; }  // uses inherited field
}
function main(): i64 {
    let d = new Dog("Rex");
    if (d.describe() != "Rex")       return 1;
```

```
    if (d.bark()     != "Rex barks") return 2;
    return 0;
}
```

Notes / current limits: the base class must be declared **before** the derived class in the file. `super` calls are not yet supported — a derived constructor initializes inherited fields directly via `this.field`. Single inheritance only; `implements` (interfaces) is type-only.

7.3 Enums

An `enum` defines a set of named integer constants. Values auto-increment from 0, or you can assign them explicitly:

```
enum Status {
    Ok = 200,
    NotFound = 404,
}
function label(s: Status): string {
    if (s == Status.Ok) return "ok";
    if (s == Status.NotFound) return "missing";
    return "other";
}
function main(): i64 {
    if (Status.Ok != 200) return 1;
    if (label(Status.NotFound) != "missing") return 2;
    return 0;
}
```

An enum value is an `i64`, so it works directly in comparisons, `switch`, and `match`. The enum name (e.g. `Status`) is a usable type annotation.

---

8. Modules

A program can span multiple `.abe` files. A file exposes declarations with `export`, and another file pulls them in with `import { ... } from "./path"` (a relative path, without the `.abe` extension). Compiling the entry file resolves and links the imported modules automatically.

8.1 Exporting and importing functions

`mathlib.abe`:

```
export function square(x: i64): i64 { return x * x; }
export function cube(x: i64): i64 { return x * x * x; }
```

`app.abe`:

```
import { square, cube } from "./mathlib";

function main(): i64 {
```

```
    if (square(5) != 25) return 1;
    if (cube(3) != 27) return 2;
    return 0;
}
```

Compile the entry file — the import is resolved for you:

```
./abec app.abe -o app
./app
```

8.2 Exporting classes and enums

Classes and enums export and import the same way:

```
// shapes.abe
export class Circle {
    r: i64;
    constructor(r: i64) { this.r = r; }
    area10(): i64 { return 3 * this.r * this.r; }
}

// status.abe
export enum HttpStatus { Ok = 200, NotFound = 404 }
```

```
import { Circle } from "./shapes";
import { HttpStatus } from "./status";

function main(): i64 {
    let c = new Circle(2);
    if (c.area10() != 12) return 1;
    if (HttpStatus.Ok != 200) return 2;
    return 0;
}
```

8.3 Sharing constants

Export module-scope `const`s to share configuration across files:

```
// config.abe
export const MAX_RETRIES: i64 = 3;
export const SERVICE_NAME: string = "abe-svc";
```

```
import { MAX_RETRIES, SERVICE_NAME } from "./config";
function main(): i64 {
    if (MAX_RETRIES != 3) return 1;
    if (SERVICE_NAME != "abe-svc") return 2;
    return 0;
}
```

> Import by name (`import { a, b } from "./m"`). Use relative paths beginning with `./`. Each name you use from a module must be `export`ed there.

The Runtime Library (Level 1)

Abe programs link a runtime that provides strings, JSON, collections, crypto, networking, and more — callable directly from `.abe` source. This part documents that surface.

---

9. Strings, JSON, and `typeof`

9.1 String methods

Strings carry a rich set of methods:

```
function main(): i64 {
    let s = "Hello, World";
    if (s.length != 12)                    return 1;
    if (s.toLowerCase() != "hello, world")  return 2;
    if (s.toUpperCase() != "HELLO, WORLD")  return 3;
    if (!s.includes("World"))               return 4;
    if (!s.startsWith("Hello"))             return 5;
    if (!s.endsWith("World"))               return 6;
    if (s.indexOf("World") != 7)            return 7;
    if (s.substring(0, 5) != "Hello")       return 8;
    if (s[0] != "H")                        return 9;   // index a character
    return 0;
}
```

Transforming and splitting:

```
function main(): i64 {
    if ("  padded  ".trim() != "padded")       return 1;
    if ("a-b-c".replace("-", "_") != "a_b_c")  return 2;
    let parts = "x,y,z".split(",");            // ["x", "y", "z"]
    if (parts.join(" ") != "x y z")            return 3;
    return 0;
}
```

Method	Result
`.length`	character count
`s[i]`	character at index `i`
`.toLowerCase()` / `.toUpperCase()`	case-folded copy
`.trim()`	surrounding whitespace removed
`.includes(sub)` / `.startsWith(p)` / `.endsWith(s)`	`bool`

Method	Result
`.indexOf(sub)`	index, or `-1`
`.substring(start, end)`	substring `[start, end)`
`.replace(old, new)`	replace occurrences
`.split(sep)`	array of pieces
`.join(sep)` *(on an array)*	join into a string

9.2 Regular expressions

Three built-ins cover pattern matching, each taking the pattern as the first argument (POSIX extended syntax):

- `regexTest(pattern, subject)` → `bool`: does the pattern match anywhere?
- `regexMatch(pattern, subject)` → the first matching substring (`""` if none).
- `regexReplace(pattern, subject, replacement)` → replace **every** match.

```
function main(): i64 {
    if (!regexTest("[0-9]+", "abc123")) return 1;
    if (regexTest("^[0-9]+$", "abc123")) return 2;       // anchored: no match
    let m = regexMatch("[0-9]+", "order 42 shipped");    // "42"
    if (m != "42") return 3;
    let r = regexReplace("[0-9]", "a1b2c3", "#");         // "a#b#c#"
    if (r != "a#b#c#") return 4;
    return 0;
}
```

9.3 Parsing JSON

`JSON.parse` turns a JSON string into a value you read with `.field` and `[i]`:

```
function main(): i64 {
    let p = JSON.parse("{\"name\":\"abe\",\"kind\":\"lang\"}");
    if (p.name != "abe")  return 1;
    if (p.kind != "lang") return 2;
    return 0;
}
```

Nested arrays and objects are indexed and accessed the same way:

```
function main(): i64 {
    let p = JSON.parse("{\"messages\":[{\"role\":\"user\",\"text\":\"hi\"}]}");
    let first = p.messages[0];   // index into a nested array
    if (first.role != "user") return 1;
    if (first.text != "hi")   return 2;
    return 0;
}
```

> Malformed JSON parses to a falsy value, so `if (!JSON.parse(src)) { ... }` safely detects bad input. Parsed leaf values are strings — compare and use them as text. **Not yet available:** `JSON.stringify`. Build JSON output with string concatenation or template literals for now.

9.4 `typeof` and `Array.isArray`

Use `typeof` and `Array.isArray` to validate shapes — especially on parsed JSON, where input structure isn't guaranteed:

```
function main(): i64 {
    if (typeof "x" != "string")    return 1;
    if (typeof 42 != "number")     return 2;
    if (typeof true != "boolean")  return 3;
    if (!Array.isArray([1, 2, 3])) return 4;
    if (Array.isArray("nope"))     return 5;
    let p = JSON.parse("{\"tags\":[\"a\",\"b\"]}");
    if (!Array.isArray(p.tags))    return 6;   // confirm tags is really an array
    return 0;
}
```

`typeof` returns `"string"`, `"number"`, `"boolean"`, `"object"` (arrays and maps), or `"undefined"` (null/absent).

---

10. Crypto and Authentication

The runtime provides production-grade primitives for password storage, token issuance, and message authentication — backed by libsodium/OpenSSL.

10.1 Password hashing (Argon2id)

`passwordHash` returns a self-contained Argon2id PHC string (algorithm, parameters, salt, and tag). `passwordVerify` checks a candidate against it in constant time. Each hash uses a fresh random salt, so the same password hashes to different strings — always store and compare the encoded hash, never the password.

```
function main(): i64 {
    let pw = "correcthorsebatterystaple";
    let enc = passwordHash(pw);                  // store `enc`
    if (!passwordVerify(pw, enc))     return 1;   // correct password
    if (passwordVerify("wrong", enc)) return 2;   // wrong rejected
    let enc2 = passwordHash(pw);
    if (enc == enc2) return 3;                    // different salt each time
    return 0;
}
```

A legacy SHA-256 verifier, `passwordVerifySha256(pw, hexHash, salt)`, exists for migrating older records — verify against it on login, then rehash with `passwordHash`.

10.2 JSON Web Tokens (HS256)

`jwtSignHS256(secret, payload)` issues a signed token; `jwtVerifyHS256(secret, token)` returns the payload on success or an **empty string** on any failure (bad signature, wrong secret, tampering, expired `exp`):

```
function main(): i64 {
    let secret = "super-secret-key";
    let payload = "{\"sub\":\"user-42\"}";
    let token = jwtSignHS256(secret, payload);
    let verified = jwtVerifyHS256(secret, token);
    if (verified != payload)                          return 1;   // round-trip
    if (jwtVerifyHS256("wrong", token).length != 0) return 2;   // wrong secret
    return 0;
}
```

An `exp` (expiry) claim in the payload is enforced automatically. To read a token's payload **without** verifying (e.g. to find which key to check), `jwtPayloadHS256(token)` returns the unverified payload.

10.3 HMAC, random tokens, and constant-time compare

```
function main(): i64 {
    // HMAC-SHA256, lowercase hex (RFC test vector)
    let h = hmacSha256Hex("key",
        "The quick brown fox jumps over the lazy dog");
    if (h != "f7bc83f430538424b13298e6aa6fb143ef4d59a14946175997479dbc2d1a3cd8")
        return 1;

    // Cryptographically-random hex tokens: N bytes -> 2N hex chars
    let session = randomTokenHex(16);   // 32 hex chars
    if (session.length != 32) return 2;

    // Constant-time comparison — use for secrets/tokens, not `==`
    if (!secureEquals("abc", "abc")) return 3;
    if (secureEquals("abc", "abd"))  return 4;
    return 0;
}
```

Function	Purpose
`passwordHash(pw)` / `passwordVerify(pw, enc)`	Argon2id password storage
`passwordVerifySha256(pw, hex, salt)`	legacy SHA-256 verify (migration)
`jwtSignHS256` / `jwtVerifyHS256` / `jwtPayloadHS256`	HS256 JWTs
`hmacSha256Hex(key, msg)`	HMAC-SHA256 → lowercase hex
`randomTokenHex(byteLen)`	secure random hex token
`secureEquals(a, b)`	constant-time string compare

---

11. Environment, Files, and Caching

11.1 Environment variables

`getEnv(name)` reads an OS environment variable. An unset variable returns the empty string (never null), so the result is always safe to use directly:

```
function main(): i64 {
    let greeting = getEnv("ABE_MANUAL_GREETING");
    let missing = getEnv("ABE_DEFINITELY_UNSET_XYZ");  // -> ""
    if (missing != "") return 1;
    return 0;
}
```

Combine with `==""` for defaults: `let port = getEnv("PORT"); if (port == "") { port = "8080"; }`.

11.2 Files

`writeFile(path, content)` writes a file, `readFile(path)` returns its contents, and `fileExists(path)` tests for presence:

```
function main(): i64 {
    let path = "/tmp/abe_demo.txt";
    writeFile(path, "persisted-value");
    if (!fileExists(path)) return 1;
    let content = readFile(path);
    if (content != "persisted-value") return 2;
    return 0;
}
```

`readFileSync` / `writeFileSync` are accepted as aliases of `readFile` / `writeFile`.

11.3 In-process caching

The runtime provides named in-process caches — handy for memoizing expensive results or HTTP responses without a Redis dependency. Entries are key/value strings with an optional TTL in seconds (`0` = no expiry):

```
function main(): i64 {
    httpCacheSet("users", "u:1", "Alice", 0);   // cache, key, value, ttl
    let hit: string = httpCacheGet("users", "u:1");
    if (hit != "Alice")                         return 1;
    if (httpCacheGet("users", "u:404") != "") return 2;   // miss -> ""
    let stats = httpCacheStats("users");
    if (stats.entries != "1")                   return 3;
    if (!httpCacheDelete("users", "u:1"))     return 4;   // true once
    if (httpCacheDelete("users", "u:1"))      return 5;   // false after
    return 0;
}
```

Function	Purpose
`httpCacheSet(cache, key, value, ttl)`	store (ttl seconds, 0 = none)
`httpCacheGet(cache, key)`	value, or `""` on miss
`httpCacheDelete(cache, key)`	`true` if it existed
`httpCacheClear(cache)`	drop all entries
`httpCacheStats(cache)`	`.entries` / `.hits` / `.misses` / `.evictions`

Multiple named caches are independent (`"users"` and `"sessions"` don't collide).

---

12. HTTP

12.1 A web server

Build a server with `server_new`, register routes, and start it with `server_listen` (which blocks, serving requests). Route handlers take a request and a response (`req: ptr, res: ptr`) and write the response:

```
function hello(req: ptr, res: ptr): void {
    response_status(res, 200);
    response_header(res, "Content-Type", "application/json");
    response_json(res, "{\"message\":\"Hello, Abe!\"}");
}
function getUser(req: ptr, res: ptr): void {
    let id: string = request_param(req, "id");      // :id path parameter
    let page: string = request_query(req, "page");   // ?page= query parameter
    response_text(res, "user " + id);
}
function createUser(req: ptr, res: ptr): void {
    let body: string = request_body(req);            // request body (POST/PUT)
    response_status(res, 201);
    response_json(res, body);
}
function main(): i64 {
    let app = server_new();
    server_port(app, 8080);
    server_cors(app, "*");
    server_get(app, "/api/hello", hello);
    server_get(app, "/api/users/:id", getUser);   // :id binds a path parameter
    server_post(app, "/api/users", createUser);
    server_listen(app);                            // blocks, serving requests
    return 0;
}
```

Server	Request (`req`)	Response (`res`)
`server_new()`	`request_param(req, name)` — `:name`	`response_status(res, code)`
`server_port(app, n)`	`request_query(req, name)` — `?`	`response_header(res, k, v)`

Server	Request (`req`)	Response (`res`)
	`name=`	
`server_cors(app, origin)`	`request_body(req)`	`response_json(res, json)`
`server_get/post/put/delete(app, path, handler)`	`request_header(req, name)`	`response_text(res, s)`
`server_use(app, mw)` — middleware	`request_method(req)` / `request_path(req)`	`response_html(res, html)`
`server_listen(app)` — start (blocking)		

Register a function value as middleware with `server_use` (it runs before route handlers and can short-circuit), and a shutdown hook with `on_shutdown`.

12.2 An HTTP client

`httpGet(url)` and `httpPost(url, body)` make requests and return a response object; read it with `httpResponseOk` and `httpResponseText`. A failed connection returns a non-OK response rather than crashing:

```
function main(): i64 {
    let resp = httpGet("https://api.example.com/v1/items");
    if (httpResponseOk(resp)) {
        let body = httpResponseText(resp);
        // ... use body ...
    }
    let created = httpPost("https://api.example.com/v1/items", "{\"name\":\"x\"}");
    return 0;
}
```

`httpPostWithHeaders(url, body, headersJson)` adds request headers (e.g. an `Authorization` bearer token) — this is what backs LLM/API integrations.

> `PUT/PATCH/DELETE` client verbs are planned.

12.3 DNS

`dnsResolveTxtJson(domain)` performs a TXT lookup and returns a JSON string (`{"ok":bool,"records":[...]}`), so results are easy to inspect:

```
function main(): i64 {
    let result: string = dnsResolveTxtJson("");   // empty domain
    if (!result.includes("\"ok\":false")) return 1;
    return 0;
}
```

> The server example above is verified **live** — the test suite boots it, sends a request to `/api/users/:id`, and checks the response — so dynamic responses built by concatenation work as shown. Broader server behaviour (middleware, streaming, graceful shutdown) is exercised by the compiler's HTTP integration test.

---

13. Database

The runtime includes a PostgreSQL client. Connect with `dbConnect`, run parameterized queries, and read results. Connections and queries are **null-safe**: a failed connection yields `null` and every downstream call degrades gracefully instead of crashing, so error handling is a simple truthy check.

13.1 Connecting

```
function main(): i64 {
    let db = dbConnect("postgres://user:pass@127.0.0.1:1/mydb");
    if (db) {                      // truthy only when the connection succeeds
        dbDisconnect(db);
    }
    // Offline here, dbConnect returns null and every call below stays null-safe.
    return 0;
}
```

13.2 Parameterized queries

Always pass user-supplied values as parameters (`$1`, `$2`, …) rather than concatenating them into SQL — this is injection-safe. Because every accessor is null-safe, you can read results without a separate connection check: with no connection `dbResultRowCount` is `0` and the loop simply doesn't run.

```
function main(): i64 {
    let db = dbConnect("postgres://127.0.0.1:1/mydb");
    let params = ["alice@example.com"];
    // Parameterized query — values bind to $1, $2, … (SQL-injection safe).
    let r = dbQueryParams(db, "SELECT id, name FROM users WHERE email = $1",
params);
    let rows = dbResultRowCount(r);     // null-safe: 0 when there is no result
    let i = 0;
    while (i < rows) {                  // with a live database, iterates matches
        let name = dbResultGet(r, i, 1);   // (row, column) -> string
        i = i + 1;
    }
    return 0;
}
```

`dbQuery(db, sql)` runs a query without parameters. Both the connection URL and SQL text may be built with concatenation or template literals (though user-supplied *values* belong in parameters, not the SQL string).

Function	Purpose
`dbConnect(url)`	connect; `null` on failure
`dbDisconnect(db)`	close the connection
`dbPing(db)`	health check (`bool`)
`dbQuery(db, sql)`	run a query → result, or `null`
`dbQueryParams(db, sql, params)`	parameterized query (`$1`…)

Function	Purpose
`dbResultRowCount(r)`	number of rows (`0` if `null`)
`dbResultGet(r, row, col)`	a cell value (string)
`dbResultRows(r)`	rows as an array of `{column: value}` maps

`dbResultRows` is convenient for JSON-shaped access — `rows[0].email` instead of `dbResultGet(r, 0, 1)`.

> The examples in this chapter are verified against the **null-safe contract** offline (no live database needed): a failed `dbConnect` returns `null` and every accessor degrades safely. Running real queries requires a reachable PostgreSQL server.

---

14. Licensing

Abe has two separately-licensed pieces:

- `abec`, the compiler **— a commercial product of** E-Tools AI Corporation**, distributed as a prebuilt binary under the** Abec Compiler EULA**. The compiler source is private. Each build runs free for a** 30-day evaluation**; once the grace window expires a license key is required. Extended trials are available — contact licensing@e-tools.ai.
- **The Abe runtime** — the library your compiled programs link. It is distributed as a **freely-redistributable binary** under the **Abe Runtime Redistribution License**: you may ship it inside the native executables `abec` produces.

In other words: you license the compiler to *build* `.abe` programs, and you may *distribute* the resulting native binaries (which embed the runtime) under the runtime's redistribution terms.

The authoritative documents live alongside the distribution (in the `abec` repository and the runtime package):

Document	Covers
`Abec-Compiler-EULA`	terms for using the `abec` compiler
`Abe-Runtime-Redistribution-License`	redistributing the linked runtime in your apps
`Abec-Licensing-FAQ`	common licensing questions

> Refer to those documents for the binding terms — this chapter is a summary, not the license itself. For questions not covered by the FAQ, contact E-Tools AI Corporation.

Appendix A — A Worked Example

This appendix ties the language and runtime together in one small program: a **task API** that serves JSON over HTTP. It uses an `enum` for domain values, a helper function, request path parameters, string-built JSON, and routing — features from across this manual in a single, runnable service.

```
// task_api.abe — a tiny JSON HTTP service
```

```
enum Priority { Low, Medium, High }

function priorityName(p: Priority): string {
    if (p == Priority.High)   return "high";
    if (p == Priority.Medium) return "medium";
    return "low";
}

function health(req: ptr, res: ptr): void {
    response_status(res, 200);
    response_json(res, "{\"status\":\"ok\",\"service\":\"tasks\"}");
}

function getTask(req: ptr, res: ptr): void {
    let id = request_param(req, "id");                // :id from the path
    let body = "{\"id\":\"" + id + "\",\"priority\":\""
             + priorityName(Priority.High) + "\"}";
    response_status(res, 200);
    response_json(res, body);
}

function main(): i64 {
    let app = server_new();
    server_port(app, 8080);
    server_cors(app, "*");
    server_get(app, "/health", health);
    server_get(app, "/tasks/:id", getTask);
    return server_listen(app);            // blocks, serving requests
}
```

Build and run it, then make requests:

```
./abec task_api.abe -o task_api
./task_api &                              # start the server
curl http://127.0.0.1:8080/health         # {"status":"ok","service":"tasks"}
curl http://127.0.0.1:8080/tasks/42       # {"id":"42","priority":"high"}
```

From here you would add `server_post` routes that read `request_body`, validate input, hash passwords with `passwordHash`, issue session tokens with `jwtSignHS256`, cache responses with `httpCacheSet`, and persist with `dbQueryParams` — each covered in Part III. The whole service compiles to a single native binary that links the runtime; there is no interpreter, package manager, or external dependency tree to deploy.

Abe Pro User Manual — Level 1 + Runtime. Level 2 (Machine Learning) ships in a later release.

PART II

Level 2: Machine Learning

1. Tensors

A **tensor** is a multi-dimensional array of numbers — the value every ML computation flows through. In Abe a tensor has a static type written `Tensor<dtype, dims...>`, e.g. `Tensor<f64, 2, 3>` is a 2×3 matrix of 64-bit floats. The element type is `f64` in this release; the dimensions are part of the type so shapes are visible in your code.

1.1 Creating tensors

`zeros`, `ones`, `full`, and `randn` build tensors of a given shape. `tensor_sum` adds every element (handy for checking results), and `tensor_numel` returns the element count.

```
function main(): i64 {
    let a: Tensor<f64, 2, 3> = ones(2, 3);   // 6 elements, all 1.0
    if (tensor_sum(a) != 6.0) return 1;
    let z: Tensor<f64, 2, 3> = zeros(2, 3);  // all 0.0
    if (tensor_sum(z) != 0.0) return 2;
    return 0;
}
```

`full(rows, cols, value)` fills a shape with one value; `randn(rows, cols)` draws from a standard normal distribution:

```
function main(): i64 {
    let f = full(2, 2, -1.5);            // 4 elements, all -1.5
    if (tensor_sum(f) != -6.0) return 1;
    if (tensor_numel(f) != 4) return 2;  // element count
    let r = randn(3, 4);                 // random-normal, shape [3,4]
    if (tensor_numel(r) != 12) return 3;
    return 0;
}
```

The type annotation is optional — `let a = ones(2, 3)` infers a tensor type — but annotating shapes documents intent and is recommended for layer/model code.

1.2 Element-wise arithmetic

`+`, `-`, `*`, and `/` operate **element-wise** on two same-shape tensors:

```
function main(): i64 {
    let a = full(2, 2, 6.0);
    let b = full(2, 2, 3.0);
    if (tensor_sum(a + b) != 36.0) return 1;   // 9.0 * 4
    if (tensor_sum(a - b) != 12.0) return 2;   // 3.0 * 4
    if (tensor_sum(a * b) != 72.0) return 3;   // 18.0 * 4 (element-wise)
    if (tensor_sum(a / b) != 8.0)  return 4;   // 2.0 * 4
```

```
    return 0;
}
```

Note that `*` is the **element-wise (Hadamard) product**, not matrix multiplication — for that, see `matmul` below.

1.3 Scaling by a scalar

Multiplying a tensor by a number scales every element. The scalar may be on either side:

```
function main(): i64 {
    let a = ones(2, 2);
    let c = a * 2.0;        // scale every element by a scalar
    if (tensor_sum(c) != 8.0) return 1;
    let d = 3.0 * a;        // scalar on either side
    if (tensor_sum(d) != 12.0) return 2;
    return 0;
}
```

Scaling is the only mixed tensor/scalar arithmetic: `+`, `-`, and `/` require two tensors of the same shape (add a `full(...)` tensor if you need to shift or divide by a constant).

1.4 Matrix multiplication

`matmul(a, b)` is the matrix product: an `[M, K]` tensor times a `[K, N]` tensor gives an `[M, N]` result.

```
function main(): i64 {
    let a: Tensor<f64, 2, 3> = ones(2, 3);
    let b: Tensor<f64, 3, 2> = ones(3, 2);
    let c: Tensor<f64, 2, 2> = matmul(a, b);  // each entry = 3.0, four entries
    if (tensor_sum(c) != 12.0) return 1;
    return 0;
}
```

1.5 Activations

The element-wise activations `relu`, `sigmoid`, and `tanh` are built in:

```
function main(): i64 {
    // relu clamps negatives to zero.
    let n = full(2, 2, -1.0);
    if (tensor_sum(relu(n)) != 0.0) return 1;
    // sigmoid(0) = 0.5 exactly, so a 4-element zero tensor sums to 2.0.
    let z = zeros(4, 1);
    if (tensor_sum(sigmoid(z)) != 2.0) return 2;
    // tanh(0) = 0.
    if (tensor_sum(tanh(z)) != 0.0) return 3;
    return 0;
}
```

The modern transformer activations `silu`, `swiglu`, `rmsnorm`, and `rope` are covered in Chapter 4 (Attention and transformer blocks).

1.6 Reshaping and inspecting

`reshape` reinterprets a tensor's elements under a new shape (the element count must match). `tensor_numel` and `tensor_sum` inspect a tensor, and `tensor_print` / `console.log(t)` dump it for debugging.

```
function main(): i64 {
    let a = ones(2, 6);             // 12 elements
    let b = reshape(a, 3, 4);       // same 12 elements, new shape
    if (tensor_numel(b) != 12) return 1;
    if (tensor_sum(b) != 12.0) return 2;   // values preserved
    return 0;
}
```

Function	Result
`zeros(r, c)` / `ones(r, c)`	a tensor filled with 0.0 / 1.0
`full(r, c, v)`	a tensor filled with `v`
`randn(r, c)`	standard-normal random values
`matmul(a, b)`	matrix product
`a + b` / `a - b` / `a * b` / `a / b`	element-wise (same shape)
`a * scalar`	scale every element
`relu(t)` / `sigmoid(t)` / `tanh(t)`	element-wise activations
`reshape(t, dims...)`	same data, new shape
`tensor_numel(t)`	element count (`i64`)
`tensor_sum(t)`	sum of all elements (`f64`)
`tensor_print(t)` / `console.log(t)`	print for debugging

---

2. Layers

A **layer** is a reusable neural-network building block: it bundles trainable **parameters** with a `forward` method that computes the layer's output. Layers are declared with `layer`, take compile-time **type parameters** (typically the input/output dimensions), instantiate with `new`, and run via `.forward(...)`.

2.1 A linear layer

This `Linear` layer holds a weight matrix and a bias vector, and its `forward` computes `weight · x + bias`. The type parameters `In`/`Out` flow into the parameter shapes:

```
layer Linear<In, Out> {
    param weight: Tensor<f64, Out, In> = ones(Out, In);
    param bias:   Tensor<f64, Out, 1>  = zeros(Out, 1);
    forward(x: Tensor<f64, In, 1>): Tensor<f64, Out, 1> {
        return matmul(this.weight, x) + this.bias;
    }
```

```
}
function main(): i64 {
    let l = new Linear<8, 4>();
    let x: Tensor<f64, 8, 1> = ones(8, 1);
    let y = l.forward(x);          // matmul(ones[4,8], ones[8,1]) = 8 per output
    if (tensor_sum(y) != 32.0) return 1;   // 4 outputs * 8
    return 0;
}
```

The pieces:

- `param name: Type = expr`** declares a trainable parameter. The initializer `expr` runs when the layer is constructed, in scope of the type parameters — so `ones(Out, In)` allocates a correctly-shaped tensor. (Common initializers: `zeros`, `ones`, `full`, `randn`.)
- `forward(args): Ret { ... }`** is the layer's computation. Inside it, `this.weight` / `this.bias` access the parameters.
- `new Linear<8, 4>()`** constructs an instance with `In=8, Out=4`.
- `l.forward(x)`** runs it.

2.2 Activations and bias

A `forward` is ordinary Abe code over tensors, so you compose `matmul`, element-wise `+`, and activations freely:

```
layer Affine<In, Out> {
    param weight: Tensor<f64, Out, In> = ones(Out, In);
    param bias:   Tensor<f64, Out, 1>  = full(Out, 1, -10.0);
    forward(x: Tensor<f64, In, 1>): Tensor<f64, Out, 1> {
        return relu(matmul(this.weight, x) + this.bias);
    }
}
function main(): i64 {
    let l = new Affine<8, 4>();
    let x: Tensor<f64, 8, 1> = ones(8, 1);
    let y = l.forward(x);          // 8 + (-10) = -2 per output, relu -> 0
    if (tensor_sum(y) != 0.0) return 1;
    return 0;
}
```

2.3 Buffers — persistent, non-trainable state

A **`buffer`** is like a `param` but is **not** updated by the optimizer: use it for fixed state a layer needs at inference time (scales, masks, running statistics). It is declared and accessed exactly like a parameter:

```
layer Scaler<N> {
    param  weight: Tensor<f64, N, N> = ones(N, N);      // trainable
    buffer factor: Tensor<f64, N, 1> = full(N, 1, 2.0); // persistent, not trained
    forward(x: Tensor<f64, N, 1>): Tensor<f64, N, 1> {
        return matmul(this.weight, x) * this.factor;
    }
}
```

```
function main(): i64 {
    let s = new Scaler<3>();
    let x: Tensor<f64, 3, 1> = ones(3, 1);
    let y = s.forward(x);          // matmul = 3 per row, * factor 2.0 = 6
    if (tensor_sum(y) != 18.0) return 1;   // 3 rows * 6
    return 0;
}
```

Shape convention. The runtime's `matmul` is 2-D, so layer inputs are written as column tensors `Tensor<f64, In, 1>` rather than 1-D vectors. This keeps every `forward` a sequence of well-defined matrix operations.

A layer may also declare **custom methods** beyond `forward` — they dispatch as `instance.method(args)` and can be called from `forward` via `this.method(args)` — and a custom `**init()` body**, which runs at construction (after `param = expr` initializers) with `this` in scope, so it can read params, call methods, and assign `this.field = expr`.

---

3. Models

A **model** composes layers (and other models) into a complete network. It is declared with `model`, holds **submodules** as fields, and — like a layer — has a `forward` method. Submodules are constructed automatically when the model is, so you never wire up children by hand.

3.1 Composing layers

This `MLP` holds two `Linear` submodules and threads its input through both. The submodule fields (`l1`, `l2`) are typed with concrete dimensions, and `this.l1.forward(...)` dispatches into each child:

```
layer Linear<In, Out> {
    param weight: Tensor<f64, Out, In> = ones(Out, In);
    param bias:   Tensor<f64, Out, 1>  = zeros(Out, 1);
    forward(x: Tensor<f64, In, 1>): Tensor<f64, Out, 1> {
        return matmul(this.weight, x) + this.bias;
    }
}
model MLP {
    l1: Linear<8, 4>;
    l2: Linear<4, 2>;
    forward(x: Tensor<f64, 8, 1>): Tensor<f64, 2, 1> {
        return this.l2.forward(this.l1.forward(x));
    }
}
function main(): i64 {
    let m = new MLP();
    let x: Tensor<f64, 8, 1> = ones(8, 1);
    let y = m.forward(x);          // l1: ones->8 per row; l2: 8*4 = 32 per row
    if (tensor_sum(y) != 64.0) return 1;   // 2 outputs * 32
    return 0;
}
```

`new MLP()` constructs the model and every submodule it owns; `m.forward(x)` runs the whole network.

3.2 Running a model with `infer`

`m.forward(x)` calls the model directly. You can also run it through an `infer` statement inside an `ai { }` block — the explicit-inference form that marks an inference boundary (and is the entry point for the inference options in later chapters):

```
layer Linear<In, Out> {
    param weight: Tensor<f64, Out, In> = ones(Out, In);
    param bias:   Tensor<f64, Out, 1>  = zeros(Out, 1);
    forward(x: Tensor<f64, In, 1>): Tensor<f64, Out, 1> {
        return matmul(this.weight, x) + this.bias;
    }
}
model MLP {
    l1: Linear<8, 4>;
    l2: Linear<4, 2>;
    forward(x: Tensor<f64, 8, 1>): Tensor<f64, 2, 1> {
        return this.l2.forward(this.l1.forward(x));
    }
}
function main(): i64 {
    let m = new MLP();
    let x: Tensor<f64, 8, 1> = ones(8, 1);
    let r: i64 = 0;
    ai {
        let y = infer m(x);        // run the model inside an ai {} block
        if (tensor_sum(y) != 64.0) { r = 1; }
    }
    return r;
}
```

`infer m(x)` invokes the model's `forward`. (`ai { }` blocks are required around `infer`; they also scope the AI/generation operations covered later.)

3.3 Arrays of submodules

Real networks stack many identical blocks. Rather than declaring `block0`, `block1`, … by hand, declare an **array submodule** with `name: [N] T`. The model constructs all `N` children, and you dispatch with `this.name[i].forward`:

```
layer Linear<In, Out> {
    param weight: Tensor<f64, Out, In> = ones(Out, In);
    param bias:   Tensor<f64, Out, 1>  = zeros(Out, 1);
    forward(x: Tensor<f64, In, 1>): Tensor<f64, Out, 1> {
        return matmul(this.weight, x) + this.bias;
    }
}
model Stack {
    blocks: [3] Linear<4, 4>;       // an array of 3 submodules
    forward(x: Tensor<f64, 4, 1>): Tensor<f64, 4, 1> {
```

```
        let h0 = this.blocks[0].forward(x);
        let h1 = this.blocks[1].forward(h0);
        let h2 = this.blocks[2].forward(h1);
        return h2;
    }
}
function main(): i64 {
    let s = new Stack();
    let x: Tensor<f64, 4, 1> = ones(4, 1);
    let y = s.forward(x);          // 4 -> 16 -> 64 per row
    if (tensor_sum(y) != 256.0) return 1;  // 4 rows * 64
    return 0;
}
```

The index may be a **runtime variable**, so you can loop over the stack — this is exactly how an N-layer transformer is applied:

```
layer Linear<In, Out> {
    param weight: Tensor<f64, Out, In> = ones(Out, In);
    param bias:   Tensor<f64, Out, 1>  = zeros(Out, 1);
    forward(x: Tensor<f64, In, 1>): Tensor<f64, Out, 1> {
        return matmul(this.weight, x) + this.bias;
    }
}
model Stack {
    blocks: [3] Linear<4, 4>;
    forward(x: Tensor<f64, 4, 1>): Tensor<f64, 4, 1> {
        let i: i64 = 0;
        let h: Tensor<f64, 4, 1> = x;
        while (i < 3) {            // a variable index works the same way
            h = this.blocks[i].forward(h);
            i = i + 1;
        }
        return h;
    }
}
function main(): i64 {
    let s = new Stack();
    let x: Tensor<f64, 4, 1> = ones(4, 1);
    let y = s.forward(x);
    if (tensor_sum(y) != 256.0) return 1;
    return 0;
}
```

Loading real weights. The models here initialize their parameters from `param = expr` initializers (`ones`, `randn`, ...). To load *pretrained* weights from disk — a `model_load(...)` per parameter, backed by a safetensors directory — see Chapter 8 (Loading models). The architecture wiring is identical; only the initializers change.

---

4. Attention and transformer blocks

This chapter covers the kernels every modern transformer (Llama, Mistral, Qwen, TinyLlama) is built from — `rmsnorm`, `silu`/`swiglu`, `rope`, and `gqa_attention` — and assembles them into one transformer block. They are built-in functions, so a block is plain Abe code over tensors.

4.1 RMSNorm

`rmsnorm(x, weight, eps)` normalizes `x` by the root-mean-square of its last axis and scales by a per-feature `weight` (a rank-1 tensor). It is the normalization used throughout Llama-family models.

```
function main(): i64 {
    // rmsnorm(x, weight, eps): x / sqrt(mean(x^2)+eps) * weight, on the last axis.
    // For x = ones and weight = ones with eps = 0, mean(x^2)=1 so the output is x.
    let x = ones(1, 4);
    let w = ones(4);                // per-feature scale, rank-1 [4]
    let n = rmsnorm(x, w, 0.0);
    if (tensor_sum(n) != 4.0) return 1;
    return 0;
}
```

4.2 SiLU and SwiGLU

`silu(x) = x · sigmoid(x)` is the activation in the feed-forward network, and `swiglu(gate, up) = silu(gate) · up` is the gated form Llama uses:

```
function main(): i64 {
    // silu(x) = x * sigmoid(x); silu(0) = 0.
    let z = zeros(8, 1);
    if (tensor_sum(silu(z)) != 0.0) return 1;
    // silu(1) = 0.731058...; four of them sum to ~2.9243.
    let s = tensor_sum(silu(ones(4, 1)));
    if (s < 2.92 || s > 2.93) return 2;
    // swiglu(gate, up) = silu(gate) * up; with both ones, == silu(1) * 1.
    let sw = tensor_sum(swiglu(ones(4, 1), ones(4, 1)));
    if (sw < 2.92 || sw > 2.93) return 3;
    return 0;
}
```

> Irrational results like `silu(1) = 0.7310…` can't be checked with `==`, so the examples assert a tolerance band (`s < 2.92 || s > 2.93`) — the same technique you'll use to test your own models.

4.3 Rotary position embeddings (RoPE)

`rope(x, start_pos, theta_base)` applies rotary position encoding to a tensor shaped `[seq, heads, head_dim]`. Query and key projections are reshaped into heads, then rotated, before attention. At position 0 the rotation is the identity:

```
function main(): i64 {
    // rope(x, start_pos, theta_base) — rotary position embedding on
    // [seq, heads, head_dim]. At position 0 the rotation is the identity.
    let x = ones(1, 1, 4);
```

```
    let r = rope(x, 0, 10000.0);
    if (tensor_sum(r) != 4.0) return 1;
    // It also applies after reshaping a projection into heads:
    let q = ones(1, 8);
    let qm = reshape(q, 1, 4, 2);    // [seq=1, heads=4, head_dim=2]
    if (tensor_sum(rope(qm, 0, 10000.0)) != 8.0) return 2;
    return 0;
}
```

4.4 Grouped-query attention

`gqa_attention(q, k, v)` is grouped-query attention: several query heads share each key/value head (the memory-saving scheme TinyLlama and Llama-3 use). The inputs are shaped `[seq, heads, head_dim]`, with more query heads than KV heads. `tensor_set_flat(t, i, v)` writes element `i` in row-major order — handy for building a tensor with known values:

```
function main(): i64 {
    // 4 query heads, 2 KV heads, head_dim 2, single decode position.
    // Heads 0,1 read KV head 0; heads 2,3 read KV head 1. With one
    // position the softmax is 1.0, so each query head's output is its
    // KV head's value row.
    let q = full(1, 4, 2, 0.0);
    tensor_set_flat(q, 0, 1.0); tensor_set_flat(q, 2, 1.0);   // q heads 0,1
    tensor_set_flat(q, 5, 1.0); tensor_set_flat(q, 7, 1.0);   // q heads 2,3
    let k = full(1, 2, 2, 0.0);
    tensor_set_flat(k, 0, 1.0); tensor_set_flat(k, 2, 1.0);
    let v = full(1, 2, 2, 0.0);
    tensor_set_flat(v, 0, 2.0); tensor_set_flat(v, 1, 3.0);   // KV0 = [2,3]
    tensor_set_flat(v, 2, 4.0); tensor_set_flat(v, 3, 5.0);   // KV1 = [4,5]
    let o = gqa_attention(q, k, v);
    // heads 0,1 -> [2,3], heads 2,3 -> [4,5]: 2*(2+3) + 2*(4+5) = 28
    if (tensor_sum(o) != 28.0) return 1;
    return 0;
}
```

4.5 A transformer block

Putting it together: a transformer block is **pre-norm attention with a residual, then a pre-norm SwiGLU feed-forward with a residual**. Here the block is a function taking its weights as arguments (Chapter 8 shows how a `model` loads those weights from disk); hidden size 8, 4 query / 2 KV heads, head_dim 2, intermediate size 16:

```
// A complete transformer block: pre-norm attention (RoPE + grouped-query
// attention) with a residual, then a pre-norm SwiGLU feed-forward with a
// residual. Hidden = 8, 4 query heads / 2 KV heads, head_dim = 2,
// intermediate = 16. Weights are passed in (Chapter 8 loads them from disk).
function block(x:         Tensor<f64, 1, 8>,
               attn_norm: Tensor<f64, 8>,
               Wq: Tensor<f64, 8, 8>, Wk: Tensor<f64, 8, 4>,
               Wv: Tensor<f64, 8, 4>, Wo: Tensor<f64, 8, 8>,
               ffn_norm:  Tensor<f64, 8>,
               Wgate: Tensor<f64, 8, 16>, Wup: Tensor<f64, 8, 16>,
```

```
                Wdown: Tensor<f64, 16, 8>): Tensor<f64, 1, 8> {
    let h  = rmsnorm(x, attn_norm, 0.0);
    let q  = matmul(h, Wq);
    let k  = matmul(h, Wk);
    let v  = matmul(h, Wv);
    let qr = rope(reshape(q, 1, 4, 2), 0, 10000.0);
    let kr = rope(reshape(k, 1, 2, 2), 0, 10000.0);
    let am = gqa_attention(qr, kr, reshape(v, 1, 2, 2));
    let ao = matmul(reshape(am, 1, 8), Wo);
    let x1 = x + ao;                        // attention residual
    let hn = rmsnorm(x1, ffn_norm, 0.0);
    let f  = swiglu(matmul(hn, Wgate), matmul(hn, Wup));
    let mo = matmul(f, Wdown);
    return x1 + mo;                         // feed-forward residual
}
function main(): i64 {
    let x  = full(1, 8, 0.5);
    let an = full(8, 1.0);
    let y = block(x, an,
        full(8, 8, 0.125), full(8, 4, 0.125), full(8, 4, 0.125), full(8, 8, 0.125),
        an, full(8, 16, 0.125), full(8, 16, 0.5), full(16, 8, 0.0625));
    let s = tensor_sum(y);
    if (s < 35.39 || s > 35.40) return 1;  // closed-form ~35.3939
    return 0;
}
```

Stack this block `N` times (Chapter 3's array submodules), add an embedding lookup at the input and a final RMSNorm + projection at the output, and you have a complete language model — exactly what Chapter 8 loads real weights into.

Function	Purpose
`rmsnorm(x, weight, eps)`	RMS normalization over the last axis
`silu(x)`	`x * sigmoid(x)` activation
`swiglu(gate, up)`	`silu(gate) * up` gated FFN activation
`rope(x, start_pos, theta)`	rotary position embedding on `[seq, heads, head_dim]`
`gqa_attention(q, k, v)`	grouped-query attention
`tensor_set_flat(t, i, v)`	write element `i` (row-major) of a tensor

---

5. Inference and generation

Once a model can run a `forward`, you can **generate** text from it. This chapter covers the KV cache (which makes autoregressive decoding fast), the `generate` statement, and the lower-level `sample` for hand-written decode loops. Generation happens inside an `ai { }` block — the explicit boundary for inference operations.

5.1 The KV cache

A **KV cache** stores the key/value tensors from previous positions so each new token only computes attention against the cache instead of re-reading the whole sequence. Allocate one with `KVCache<dtype, layers, max_seq, head_dim>`; `kv_cache_seq_len` reports how many positions it holds:

```
function main(): i64 {
    let r: i64 = 0;
    ai {
        // KVCache<dtype, num_layers, max_seq, head_dim>
        let cache = KVCache<f64, 2, 16, 4>(maxBatchSize: 1);
        if (kv_cache_seq_len(cache) != 0) { r = 1; }   // empty to start
    }
    return r;
}
```

5.2 Generating tokens

`generate model(prompt) with { … }` runs an autoregressive decode loop: it calls the model's `forward`, samples a token, feeds it back, and repeats up to `maxNewTokens`. `temperature: 0.0` selects greedy decoding (argmax). The result is a tensor of token ids.

```
layer Tiny<In, Vocab> {
    param weight: Tensor<f64, Vocab, In> = ones(Vocab, In);
    param bias:   Tensor<f64, Vocab, 1>  = zeros(Vocab, 1);
    forward(x: Tensor<f64, In, 1>): Tensor<f64, Vocab, 1> {
        return matmul(this.weight, x) + this.bias;
    }
}
function main(): i64 {
    let lm = new Tiny<8, 5>();
    let prompt: Tensor<f64, 8, 1> = ones(8, 1);
    let r: i64 = 0;
    ai {
        // Greedy generation (temperature 0 = argmax). Uniform logits -> token 0.
        let toks = generate lm(prompt) with {
            maxNewTokens: 5;
            temperature:  0.0;
        };
        if (tensor_numel(toks) != 5) { r = 1; }   // five new tokens
        if (tensor_sum(toks) != 0.0) { r = 2; }   // all token id 0
    }
    return r;
}
```

5.3 Generating with a KV cache

For a cache-aware model, pass the cache to `generate` with `cache:`. The model's `forward` takes the cache as a parameter and appends each step's key/value with `kv_cache_append` / `kv_cache_advance`:

```
layer Cached {
    param weight: Tensor<f64, 5, 8> = ones(5, 8);
```

```
    forward(x: Tensor<f64, 8, 1>,
            cache: KVCache<f64, 1, 16, 4>): Tensor<f64, 5, 1> {
        // A real layer projects K/V here; this commits one position per step.
        kv_cache_append(cache, 0, ones(4, 1), ones(4, 1));
        kv_cache_advance(cache);
        return matmul(this.weight, x);
    }
}
function main(): i64 {
    let lm = new Cached();
    let prompt: Tensor<f64, 8, 1> = ones(8, 1);
    let r: i64 = 0;
    ai {
        let cache = KVCache<f64, 1, 16, 4>(maxBatchSize: 1);
        let toks = generate lm(prompt) with {
            maxNewTokens: 3;
            temperature:  0.0;
            cache:        cache;       // thread the cache through the decode loop
        };
        if (tensor_numel(toks) != 3) { r = 1; }
        if (kv_cache_seq_len(cache) != 3) { r = 2; }   // 3 positions cached
    }
    return r;
}
```

5.4 Sampling and manual decode loops

`generate` is convenient, but you can drive decoding yourself for full control. `sample(logits, temperature, topK, topP, seed)` turns a logits tensor into a token id — greedy when `temperature` is 0, otherwise temperature/top-k/top-p sampling:

```
function main(): i64 {
    // sample(logits, temperature, topK, topP, seed) -> token id.
    // Temperature 0 is greedy argmax; uniform logits pick index 0.
    let logits = ones(5, 1);
    let tok = sample(logits, 0.0, 0, 1.0, 0);
    if (tok != 0) return 1;
    return 0;
}
```

Putting it in a loop — call `forward`, `sample`, repeat — is exactly what `generate` does under the hood:

```
layer Cached {
    param weight: Tensor<f64, 5, 8> = ones(5, 8);
    forward(x: Tensor<f64, 8, 1>,
            cache: KVCache<f64, 1, 16, 4>): Tensor<f64, 5, 1> {
        kv_cache_append(cache, 0, ones(4, 1), ones(4, 1));
        kv_cache_advance(cache);
        return matmul(this.weight, x);
    }
}
```

```
function main(): i64 {
    let lm = new Cached();
    let prompt: Tensor<f64, 8, 1> = ones(8, 1);
    let r: i64 = 0;
    ai {
        // The lower-level alternative to `generate`: drive the loop yourself,
        // calling forward and sampling each step.
        let cache = KVCache<f64, 1, 16, 4>(maxBatchSize: 1);
        let i: i64 = 0;
        while (i < 3) {
            let logits: Tensor<f64, 5, 1> = lm.forward(prompt, cache);
            let tok: i64 = sample(logits, 0.0, 0, 1.0, 0);
            if (tok != 0) { r = 1; }
            i = i + 1;
        }
        if (kv_cache_seq_len(cache) != 3) { r = 2; }
    }
    return r;
}
```

Function / form	Purpose
`KVCache<dt, layers, max_seq, head_dim>(maxBatchSize: n)`	allocate a KV cache
`kv_cache_seq_len(c)`	positions currently cached (`i64`)
`kv_cache_append(c, layer, k, v)` / `kv_cache_advance(c)`	commit a position
`generate m(prompt) with { maxNewTokens; temperature; cache; }`	autoregressive decode
`sample(logits, temp, topK, topP, seed)`	logits → token id

---

6. Tokenization

A language model works on **token ids**, not text. A **tokenizer** converts between the two: `encode` turns a string into a tensor of ids, and `decode` turns ids back into a string. Abe's runtime has a byte-level **BPE** tokenizer (the scheme Llama, GPT-2, and friends use), loaded from a `tokenizer.json`.

6.1 Encoding and decoding

`tokenizer_load(path)` activates a tokenizer (returns 0 on success); `vocab_size()` reports the vocabulary size; `encode` / `decode` round-trip text; `decode_one(id)` decodes a single id; and `tokenizer_unload()` releases it. The example writes a tiny `tokenizer.json` inline so it is self-contained — a real program points `tokenizer_load` at a model's own tokenizer file.

```
function main(): i64 {
    // A real program loads a model's tokenizer.json; this writes a tiny
    // byte-level BPE fixture inline so the example is self-contained.
    mkdir("/tmp/abe_ml6_demo");
```

```
    write_text("/tmp/abe_ml6_demo/tokenizer.json",
        "{\"model\":{\"type\":\"BPE\",\"vocab\":
{\"h\":0,\"e\":1,\"l\":2,\"o\":3,\"ll\":4,\"lo\":5,\"he\":6,\"hell\":7,\"hello\":8,\
"\\u0120\":9,\"w\":10,\"r\":11,\"d\":12,\"o\\u0120\":13},\"merges\":[\"o \\
u0120\",\"l l\",\"l o\",\"h e\",\"he ll\",\"hell o\"]}}");

    let r: i64 = 0;
    if (tokenizer_load("/tmp/abe_ml6_demo/tokenizer.json") != 0) { r = 1; }
    if (vocab_size() != 14) { r = 2; }

    let ids = encode("hello");          // BPE merges all the way to token 8
    if (tensor_sum(ids) != 8.0) { r = 3; }
    if (decode(ids) != "hello") { r = 4; }   // round-trips back to text
    if (decode_one(6) != "he") { r = 5; }    // a single id -> its token

    tokenizer_unload();
    return r;
}
```

`encode` returns an ordinary tensor of ids, so it feeds straight into a model's `forward` / `generate`; `decode` takes the generated id tensor back to text.

6.2 Word boundaries

Byte-level BPE **pre-tokenizes** on word boundaries before merging, so a merge never spans two words. Encoding `"hello world"` keeps `hello` intact and starts a fresh run at the space — it does not merge the trailing `o` of *hello* with the leading space of *world*:

```
function main(): i64 {
    mkdir("/tmp/abe_ml6_demo2");
    write_text("/tmp/abe_ml6_demo2/tokenizer.json",
        "{\"model\":{\"type\":\"BPE\",\"vocab\":
{\"h\":0,\"e\":1,\"l\":2,\"o\":3,\"ll\":4,\"lo\":5,\"he\":6,\"hell\":7,\"hello\":8,\
"\\u0120\":9,\"w\":10,\"r\":11,\"d\":12,\"o\\u0120\":13},\"merges\":[\"o \\
u0120\",\"l l\",\"l o\",\"h e\",\"he ll\",\"hell o\"]}}");

    let r: i64 = 0;
    tokenizer_load("/tmp/abe_ml6_demo2/tokenizer.json");
    // Words are split before BPE, so merges never cross a word boundary:
    // "hello world" -> [hello=8, " "=9, w=10, o=3, r=11, l=2, d=12] = 55.
    let ids = encode("hello world");
    if (tensor_sum(ids) != 55.0) { r = 1; }
    if (decode(ids) != "hello world") { r = 2; }
    tokenizer_unload();
    return r;
}
```

The runtime tokenizer handles the full GPT-2 byte ↔ unicode mapping and UTF-8, so it round-trips multilingual text, not just ASCII.

Function	Purpose
`tokenizer_load(path)`	activate a `tokenizer.json` (returns 0 on success)
`vocab_size()`	vocabulary size (`i64`)
`encode(text)`	text → tensor of token ids
`decode(ids)`	id tensor → string
`decode_one(id)`	a single id → its token string
`tokenizer_unload()`	release the active tokenizer

---

7. Serving

Generating for one prompt is Chapter 5; **serving** runs many requests efficiently. This chapter covers the request **scheduler** (continuous batching), the **paged** KV cache and fused `paged_attention`, memory budgeting with page pools, and reproducible (deterministic) serving.

> **Generic-type spacing.** Write a space before = when a type ends in >: `let c: KVCache<f64, 1, 16, 4> = …`, not `…4>=…`. Without the space the lexer reads `>=` as a single token. All examples here follow this convention.

7.1 The scheduler

`scheduler_new(policy, maxBatch, maxPending)` creates a request scheduler (policy 0 = FIFO). You `submit` requests (each with its own KV cache and token budget), then loop: `pick` a ready request, run one `forward` step on its cache, `sample`, and `record` the token. `pending` drives the loop; `num_finished` and `get_tokens` read the results.

```
layer Cached {
    param weight: Tensor<f64, 5, 8> = ones(5, 8);
    forward(x: Tensor<f64, 8, 1>,
            cache: KVCache<f64, 1, 16, 4>): Tensor<f64, 5, 1> {
        kv_cache_append(cache, 0, ones(4, 1), ones(4, 1));
        kv_cache_advance(cache);
        return matmul(this.weight, x);
    }
}
function main(): i64 {
    let lm = new Cached();
    let prompt: Tensor<f64, 8, 1> = ones(8, 1);
    let r: i64 = 0;
    ai {
        let ca = KVCache<f64, 1, 16, 4>(maxBatchSize: 1);
        let cb = KVCache<f64, 1, 16, 4>(maxBatchSize: 1);
        // scheduler_new(policy, maxBatch, maxPending): policy 0 = FIFO.
        let sched = scheduler_new(0, 4, 8);
        let ra = scheduler_submit(sched, ca, 2);   // request A: 2 new tokens
        let rb = scheduler_submit(sched, cb, 3);   // request B: 3 new tokens
        // Drive the decode loop: pick a request, run one step, record its token.
        while (scheduler_pending(sched) > 0) {
```

```
            let req = scheduler_pick(sched);
            if (req < 0) { break; }
            let logits = lm.forward(prompt, scheduler_get_cache(sched, req));
            scheduler_record(sched, req, sample(logits, 0.0, 0, 1.0, 0));
        }
        if (scheduler_num_finished(sched) != 2) { r = 1; }
        if (tensor_numel(scheduler_get_tokens(sched, ra)) != 2) { r = 2; }
        if (tensor_numel(scheduler_get_tokens(sched, rb)) != 3) { r = 3; }
    }
    return r;
}
```

7.2 Paged attention

A **paged** KV cache stores tokens in fixed-size pages, allocated lazily as the sequence grows — the layout that lets a server pack many variable-length sequences into one pool. Enable it with `pagedKV: true, blockSize: n`. `paged_attention(cache, layer, q)` is a single fused op that walks the page table directly instead of gathering a contiguous K/V tensor:

```
layer Attn {
    param Wq: Tensor<f64, 4, 4> = ones(4, 4);
    param Wk: Tensor<f64, 4, 4> = ones(4, 4);
    param Wv: Tensor<f64, 4, 4> = ones(4, 4);
    param Wo: Tensor<f64, 5, 4> = ones(5, 4);
    forward(x: Tensor<f64, 4, 1>,
            cache: KVCache<f64, 1, 16, 4>): Tensor<f64, 5, 1> {
        kv_cache_append(cache, 0, matmul(this.Wk, x), matmul(this.Wv, x));
        kv_cache_advance(cache);
        // One fused op walks the paged cache — no contiguous K/V is built.
        let attn_t = paged_attention(cache, 0, matmul(this.Wq, x));
        return matmul(this.Wo, attn_t);
    }
}
function main(): i64 {
    let lm = new Attn();
    let prompt: Tensor<f64, 4, 1> = ones(4, 1);
    let r: i64 = 0;
    ai {
        // A paged cache stores tokens in fixed-size pages, allocated lazily.
        let cache = KVCache<f64, 1, 16, 4>(maxBatchSize: 1, pagedKV: true,
blockSize: 2);
        let toks = generate lm(prompt) with { maxNewTokens: 4; temperature: 0.0;
cache: cache; };
        if (tensor_numel(toks) != 4) { r = 1; }
        if (kv_cache_seq_len(cache) != 4) { r = 2; }
    }
    return r;
}
```

7.3 Page pools and variable-length serving

`poolPages` budgets the cache's memory **below** the worst case. When the pool is exhausted, you reclaim a finished sequence's pages with `kv_cache_release`, and the freed pages serve the next request — the substrate for packing many variable-length sequences into bounded memory:

```
function main(): i64 {
    let r: i64 = 0;
    ai {
        // poolPages budgets memory below the worst case (4 pages here): only
        // 2 pages exist, so a long sequence must release pages between rounds.
        let cache = KVCache<f64, 1, 8, 4>(maxBatchSize: 1, pagedKV: true, blockSize:
2, poolPages: 2);
        let i: i64 = 0;
        while (i < 4) { kv_cache_append(cache, 0, ones(4,1), ones(4,1));
kv_cache_advance(cache); i = i + 1; }
        if (kv_cache_seq_len(cache) != 4) { r = 1; }
        kv_cache_release(cache, 0);              // return this row's pages to the
pool
        if (kv_cache_seq_len(cache) != 0) { r = 2; }
        let j: i64 = 0;
        while (j < 4) { kv_cache_append(cache, 0, ones(4,1), ones(4,1));
kv_cache_advance(cache); j = j + 1; }
        if (kv_cache_seq_len(cache) != 4) { r = 3; }   // pages reused for the next
round
    }
    return r;
}
```

7.4 Deterministic serving

For reproducible output (tests, debugging, audits), `set_deterministic(seed)` fixes the sampling RNG so the same seed and config produce identical tokens:

```
layer Tiny {
    param weight: Tensor<f64, 5, 8> = ones(5, 8);
    forward(x: Tensor<f64, 8, 1>): Tensor<f64, 5, 1> { return matmul(this.weight,
x); }
}
function main(): i64 {
    let lm = new Tiny();
    let prompt: Tensor<f64, 8, 1> = ones(8, 1);
    let r: i64 = 0;
    ai {
        // Same seed + config => identical sampled output (reproducible serving).
        set_deterministic(42);
        let a = generate lm(prompt) with { maxNewTokens: 4; temperature: 1.0; topK:
5; seed: 0; };
        set_deterministic(42);
        let b = generate lm(prompt) with { maxNewTokens: 4; temperature: 1.0; topK:
5; seed: 0; };
        if (tensor_sum(a) != tensor_sum(b)) { r = 1; }
```

```
        if (tensor_numel(a) != 4) { r = 2; }
    }
    return r;
}
```

The runtime also exposes a telemetry log (`telemetry_clear` / `telemetry_emit` / `telemetry_count` / `telemetry_dump`) for recording per-request events.

Function	Purpose
`scheduler_new(policy, maxBatch, maxPending)`	create a scheduler (0 = FIFO)
`scheduler_submit(s, cache, maxNew)`	enqueue a request → request id
`scheduler_pending(s)` / `scheduler_pick(s)`	loop control / next ready request
`scheduler_get_cache(s, req)`	the request's KV cache
`scheduler_record(s, req, tok)`	append a sampled token
`scheduler_num_finished(s)` / `scheduler_get_tokens(s, req)`	results
`KVCache<…>(…, pagedKV: true, blockSize: n, poolPages: p)`	paged cache + pool budget
`paged_attention(cache, layer, q)`	fused page-aware attention
`kv_cache_release(cache, row)`	return a row's pages to the pool
`set_deterministic(seed)`	fix the sampling RNG

---

8. Loading models

Every model so far initialized its parameters from `ones`/`randn`/`full`. This chapter loads **real pretrained weights** from disk in the **safetensors** format — the same files HuggingFace ships — so the architectures from Chapters 2–4 run with actual weights.

8.1 Saving and loading tensors

`save_safetensors(path, name, tensor)` writes a tensor; `load_safetensors(path, name)` reads it back. A layer loads its weight by calling `load_safetensors` in the parameter initializer — it runs when the layer is constructed, so the parameter holds the on-disk tensor by the time `new` returns:

```
layer Loaded {
    // The initializer reads the weight from disk when the layer is constructed.
    param weight: Tensor<f64, 5, 8> =
        load_safetensors("/tmp/abe_ml8_w.safetensors", "weight");
    forward(x: Tensor<f64, 8, 1>): Tensor<f64, 5, 1> {
        return matmul(this.weight, x);
    }
}
function main(): i64 {
    let w = full(5, 8, 0.5);
    save_safetensors("/tmp/abe_ml8_w.safetensors", "weight", w);
```

```
    let lm = new Loaded();             // __init reads the weight back
    let y = lm.forward(ones(8, 1));
    if (tensor_sum(y) != 20.0) return 1;   // 5 rows * (8 * 0.5) = 20
    return 0;
}
```

8.2 Model directories

Real LLMs ship as a **directory**: one or more `*.safetensors` shards, a `model.safetensors.index.json` mapping each tensor name to its shard, and a `config.json`. `model_open(dir)` opens it; `model_config_int(key, default)` reads config values; `model_load(name)` fetches a tensor (routing to the right shard); `model_close()` releases it.

```
function main(): i64 {
    // A real program points model_open() at a downloaded HF directory. This
    // builds a tiny two-shard one inline so the example is self-contained.
    mkdir("/tmp/abe_ml8_dir");
    save_safetensors("/tmp/abe_ml8_dir/s1.safetensors",
                     "model.layers.0.q_proj.weight", full(2, 3, 0.5));
    save_safetensors("/tmp/abe_ml8_dir/s2.safetensors",
                     "model.layers.0.k_proj.bias", full(4, 1, 0.25));
    write_text("/tmp/abe_ml8_dir/model.safetensors.index.json",
        "{\"weight_map\":
{\"model.layers.0.q_proj.weight\":\"s1.safetensors\",\"model.layers.0.k_proj.bias\":
\"s2.safetensors\"}}");
    write_text("/tmp/abe_ml8_dir/config.json",
        "{\"hidden_size\":512,\"num_hidden_layers\":4}");

    let r: i64 = 0;
    if (model_open("/tmp/abe_ml8_dir") != 0) { r = 1; }
    if (model_config_int("hidden_size", -1) != 512) { r = 2; }
    if (model_config_int("missing", 99) != 99) { r = 3; }   // default when absent
    // Two tensors that live on different shards — proves shard routing.
    if (tensor_sum(model_load("model.layers.0.q_proj.weight")) != 3.0) { r = 4; }
    if (tensor_sum(model_load("model.layers.0.k_proj.bias")) != 1.0) { r = 5; }
    model_close();
    return r;
}
```

A `model` (or `layer`) loads its weights by calling `model_load("<name>")` in each parameter's initializer — combine this with the architecture from Chapters 3–4 and you have a runnable LLM. Pointed at a real TinyLlama / Llama / Qwen directory, the same `model_load("model.layers.0.q_proj.weight")` names resolve the actual weights.

8.3 Stacking N blocks with `model_load_at`

A transformer has many identical blocks whose weights differ only by a layer index in the name (`layer_0_w`, `layer_1_w`, …). In an array submodule (Chapter 3), each child knows its position via the `__index__` identifier, and `model_load_at(prefix, __index__, suffix)` builds the per-index weight name at construction time:

```
layer Indexed {
    // __index__ is this layer's position in its array; model_load_at builds the
    // weight name "layer_<i>_w" so each stacked block loads its own weight.
    param weight: Tensor<f64, 2, 1> = model_load_at("layer_", __index__, "_w");
    forward(x: Tensor<f64, 2, 1>): Tensor<f64, 2, 1> {
        return x + this.weight;
    }
}
model Stack {
    blocks: [3] Indexed;
    forward(x: Tensor<f64, 2, 1>): Tensor<f64, 2, 1> {
        return
this.blocks[2].forward(this.blocks[1].forward(this.blocks[0].forward(x)));
    }
}
function main(): i64 {
    mkdir("/tmp/abe_ml8_idx");
    save_safetensors("/tmp/abe_ml8_idx/w0.safetensors", "layer_0_w", full(2, 1,
1.0));
    save_safetensors("/tmp/abe_ml8_idx/w1.safetensors", "layer_1_w", full(2, 1,
2.0));
    save_safetensors("/tmp/abe_ml8_idx/w2.safetensors", "layer_2_w", full(2, 1,
3.0));
    write_text("/tmp/abe_ml8_idx/model.safetensors.index.json",
        "{\"weight_map\":
{\"layer_0_w\":\"w0.safetensors\",\"layer_1_w\":\"w1.safetensors\",\"layer_2_w\":\"w
2.safetensors\"}}");
    write_text("/tmp/abe_ml8_idx/config.json", "{}");

    model_open("/tmp/abe_ml8_idx");
    let s = new Stack();
    let y = s.forward(zeros(2, 1));        // 0 + w0 + w1 + w2 = (1+2+3) per element
    model_close();
    if (tensor_sum(y) != 12.0) return 1;   // (1+2+3) over 2 elements
    return 0;
}
```

This is how an N-layer model loads N blocks' worth of distinct weights from one directory without hand-writing each name.

Function	Purpose
`save_safetensors(path, name, t)`	write a tensor to a `.safetensors` file
`load_safetensors(path, name)`	read a tensor from a file
`model_open(dir)`	open a model directory (returns 0 on success)
`model_config_int(key, default)`	read an `i64` from `config.json`
`model_count()`	number of tensors in the index
`model_load(name)`	load a tensor by name (shard-routed)

Function	Purpose
`model_load_at(prefix, i, suffix)`	load `prefix + i + suffix` (per-index weights)
`__index__`	a layer's position within its `[N] T` array
`model_close()`	release the open model

---

9. Training

Abe trains models natively: the `train` statement drives a full loop — forward, backward (reverse-mode autograd), and an optimizer step — over a dataloader, with callbacks for logging and periodic evaluation. This chapter covers training a model from scratch and parameter-efficient fine-tuning with LoRA.

9.1 The training loop

`train model on dataloader { … }` runs the loop. Inside the block you set `epochs`/`batchSize`, choose an `optimizer` (AdamW or SGD with its hyperparameters), optionally `gradientClip`, and attach **callbacks** with `on <event>(args) { … }` — `step`, `epochEnd`, `trainEnd`, and `validation`. An `eval { … }` block runs periodic validation on a held-out dataloader. After training, `export` writes the weights to safetensors.

The dataset and dataloader are runtime helpers, declared with `extern` (Level 1, Chapter 8) and used here to feed the loop:

```
extern function abe_dataset_new(): ptr;
extern function abe_dataset_demo_tokens(ds: ptr, mod_v: i64, count: i64): void;
extern function abe_dataloader_new(ds: ptr, b: i64, s: i64, shuf: bool, seed: i64):
ptr;

layer Embed<V, D> {
    param table: Tensor<f64, V, D>;
    forward(ids: Tensor<f64>): Tensor<f64> { return embedding_lookup(this.table,
ids); }
}
layer Proj<D, V> {
    param weight: Tensor<f64, D, V>;
    forward(x: Tensor<f64>): Tensor<f64> { return matmul(x, this.weight); }
}
model Tiny<V, D> {
    embed: Embed<V, D>;
    proj:  Proj<D, V>;
    forward(ids: Tensor<f64>): Tensor<f64> { return
this.proj.forward(this.embed.forward(ids)); }
}
function main(): i64 {
    let m = new Tiny<8, 4>();
    // A tiny demo dataset + dataloader (runtime helpers via extern).
    const ds = abe_dataset_new();
    abe_dataset_demo_tokens(ds, 8, 64);
    const dl = abe_dataloader_new(ds, 2, 4, false, 0);
```

```
    ai {
        train m on dl {
            epochs:    2;
            batchSize: 2;
            optimizer: AdamW { lr: 1e-2; eps: 1e-8; weightDecay: 0.0; };
            gradientClip: { maxNorm: 1.0 };
            on step(s, loss) if s % 4 === 0 { console.log("step", s); }
            on trainEnd(m) { console.log("done"); }
        }
        export m { format: safetensors; path: "/tmp/abe_ml9_train.safetensors"; };
    }
    console.log("trained");
    return 0;
}
```

The loop wires up automatically: the autograd tape records each op in `forward`, the backward pass computes gradients, `gradientClip` bounds them, and the optimizer updates every registered parameter. Autograd covers the transformer ops from Chapter 4 (matmul, add, the activations, softmax, RMSNorm, attention, cross-entropy), so real blocks are trainable — not just this toy.

`train` lives inside an `ai { }` block. The callback events are `step(s, loss)`, `epochEnd(e, metrics)`, `trainEnd(metrics)`, and `validation(s, metrics)`; an `eval { data: …; interval: …; metrics: […]; }` block configures the periodic validation that fires the `validation` callback.

9.2 LoRA fine-tuning

LoRA (Low-Rank Adaptation) fine-tunes a large model by freezing its pretrained weights and training only small adapter matrices — far fewer parameters. You write the adapter arithmetic in `forward`; the `@lora(...)` decorator on the `train` statement tells the optimizer to **freeze the `weight` of each target layer and train only the adapters**:

```
extern function abe_dataset_new(): ptr;
extern function abe_dataset_demo_tokens(ds: ptr, mod_v: i64, count: i64): void;
extern function abe_dataloader_new(ds: ptr, b: i64, s: i64, sh: bool, seed: i64):
ptr;

layer Embed<V, D> {
    param table: Tensor<f64, V, D>;
    forward(ids: Tensor<f64>): Tensor<f64> { return embedding_lookup(this.table,
ids); }
}
layer LoraLinear<In, Out, R> {
    param weight: Tensor<f64, In, Out>;   // frozen base (excluded by @lora)
    param a:      Tensor<f64, In, R>;      // LoRA adapter A (trained)
    param b:      Tensor<f64, R, Out>;     // LoRA adapter B (trained)
    forward(x: Tensor<f64>): Tensor<f64> {
        const base  = matmul(x, this.weight);
        const delta = matmul(matmul(x, this.a), this.b);
        return base + delta;
    }
}
```

```
model Tiny<V, D, R> {
    embed: Embed<V, D>;
    proj:  LoraLinear<D, V, R>;
    forward(ids: Tensor<f64>): Tensor<f64> { return
this.proj.forward(this.embed.forward(ids)); }
}
function main(): i64 {
    let m = new Tiny<8, 4, 2>();
    const ds = abe_dataset_new();
    abe_dataset_demo_tokens(ds, 8, 64);
    const dl = abe_dataloader_new(ds, 2, 4, false, 0);
    ai {
        // @lora freezes each target layer's `weight` and trains only the adapters.
        @lora(rank: 2, alpha: 4, dropout: 0.0, targetLayers: ["proj"])
        train m on dl {
            epochs: 1;
            optimizer: AdamW { lr: 1e-2; eps: 1e-8; weightDecay: 0.0; };
        }
    }
    console.log("lora-ok");
    return 0;
}
```

The forward arithmetic (`base + delta`) is yours; `@lora` only controls **which parameters receive gradients** — the `weight` slots in `targetLayers` are frozen, the adapters `a`/`b` are trained.

Form	Purpose
`train m on dl { … }`	run the training loop
`optimizer: AdamW { lr; eps; weightDecay; }` / `SGD { … }`	choose the optimizer
`gradientClip: { maxNorm: … }`	clip gradient norm
`on step(s, loss) if … { … }`	per-step callback (optional guard)
`on epochEnd / trainEnd / validation(…) { … }`	lifecycle callbacks
`eval { data; interval; metrics; }`	periodic validation config
`export m { format: safetensors; path: …; }`	save trained weights
`@lora(rank, alpha, dropout, targetLayers)`	freeze base, train adapters

> **Scope.** Training is CPU `f64`, with reverse-mode autograd over the transformer op set above and the AdamW/SGD optimizers. Distributed training and mixed-precision (`@amp`) are not yet available.

---

10. Quantization

A trained model's weights are `f64` — 8 bytes per number. **Quantization** shrinks them by storing each weight in far fewer bits plus a shared scale, which is what lets a large model fit in a fraction of the memory. Abe

quantizes **weights** (the dominant cost) and **dequantizes on the fly** inside `matmul`, so the math stays `f64` and the rest of your model code is unchanged.

10.1 Quantizing a model

`quantize(model, { bits: 8 })` (or `bits: 4`) converts every parameter to block-quantized form **in place** — the `f64` data is freed and replaced by a compact int8 / 4-bit payload with one scale per block of 32. A `forward` afterwards transparently dequantizes; for weights whose values are uniform, the result is unchanged:

```
layer Linear {
    param weight: Tensor<f64, 64, 64> = full(64, 64, 0.5);
    forward(x: Tensor<f64, 64, 1>): Tensor<f64, 64, 1> { return matmul(this.weight,
x); }
}
layer Weights {                          // a peek layer, just to measure storage
    param w: Tensor<f64, 64, 64> = full(64, 64, 0.5);
    forward(): Tensor<f64, 64, 64> { return this.w; }
}
function main(): i64 {
    // Correctness: a quantized layer's forward is unchanged (uniform = lossless).
    let net = new Linear();
    let x: Tensor<f64, 64, 1> = ones(64, 1);
    let before = tensor_sum(net.forward(x));         // 64 rows * (64*0.5) = 2048
    quantize(net, { bits: 8 });
    if (tensor_sum(net.forward(x)) != before) return 1;

    // Storage for a 64x64 weight: f64 -> Q8_0 -> Q4_0.
    let m8 = new Weights();
    if (tensor_nbytes(m8.forward()) != 32768) return 2;   // f64
    quantize(m8, { bits: 8 });
    if (tensor_nbytes(m8.forward()) != 5120) return 3;    // Q8_0 (~6.4x)

    let m4 = new Weights();
    quantize(m4, { bits: 4 });
    if (tensor_nbytes(m4.forward()) != 3072) return 4;    // Q4_0 (~10.7x)
    return 0;
}
```

`tensor_nbytes(t)` reports a tensor's storage. The two formats:

Format	`bits`	Storage	Reduction
`f64` (default)	—	8 B/elem	1×
Q8_0 (8-bit)	`8`	~1.25 B/elem	~6.4×
Q4_0 (4-bit)	`4`	~0.75 B/elem	~10.7×

Fewer bits means less memory but more rounding error — 8-bit is near-lossless, 4-bit trades accuracy for the bigger saving.

10.2 Saving and loading quantized weights

Quantized weights round-trip through disk **compact** — `save_safetensors` writes the int8/4-bit payload (not `f64`), and `load_safetensors` reads it back without re-expanding, so a model loaded from disk keeps its small footprint:

```
layer Src {
    param w: Tensor<f64, 64, 64> = full(64, 64, 0.5);
    forward(): Tensor<f64, 64, 64> { return this.w; }
}
layer Dst {
    param w: Tensor<f64, 64, 64> = load_safetensors("/tmp/abe_ml10.st", "w");
    forward(x: Tensor<f64, 64, 1>): Tensor<f64, 64, 1> { return matmul(this.w, x); }
}
function main(): i64 {
    let s = new Src();
    quantize(s, { bits: 8 });
    save_safetensors("/tmp/abe_ml10.st", "w", s.forward());   // persist the
quantized weight
    let d = new Dst();                                          // loads it back,
still compact
    if (tensor_sum(d.forward(ones(64, 1))) != 2048.0) return 1;
    return 0;
}
```

`export model { format: safetensors; path: … }` (Chapter 9) likewise writes a quantized model's weights compactly when the model has been quantized.

Function	Purpose	
`quantize(model, { bits: 8 \	4 })`	quantize every weight in place (Q8_0 / Q4_0)
`tensor_nbytes(t)`	a tensor's storage in bytes (`i64`)	
`save_safetensors` / `load_safetensors`	persist / load weights, quantized or `f64`	

> **Scope.** Quantization is **weight-only** and for **inference** — training stays `f64`. It is CPU, and loads/saves Abe's own safetensors format; loading off-the-shelf GGUF models, 4-bit super-block formats (Q4_K), and integer matmul for speed are not yet available (see the appendix).

---

11. Mixed precision

Quantization (Chapter 10) shrinks weights to a few bits. **Mixed precision** is the gentler, standard middle ground: store weights as **16-bit floats** instead of `f64`. Two formats are available — **bf16** (a 7-bit mantissa with `f32`'s full exponent range) and **f16** (IEEE-754 half: a 10-bit mantissa but a narrower range). Both are **a quarter** the size of `f64`. As with quantization, Abe widens a 16-bit weight back to `f64` **on the fly** inside `matmul`, so the kernels and the rest of your code are unchanged.

11.1 Casting tensors and models

`to_bf16` / `to_f16` convert a tensor **in place** to 16-bit storage; `to_f64` widens it back. Applied to a **model**, they walk every parameter at once:

```
layer Net {
    param w: Tensor<f64, 4, 64> = full(4, 64, 0.5);
    forward(x: Tensor<f64, 64, 1>): Tensor<f64, 64, 1> { return matmul(this.w, x); }
}
function main(): i64 {
    // Tensor-level: 256-elem weight, f64 2048 B -> bf16 512 B (4x). 0.5 is exact.
    let a = full(4, 64, 0.5);
    if (tensor_nbytes(a) != 2048) return 1;
    to_bf16(a);
    if (tensor_nbytes(a) != 512) return 2;
    if (tensor_sum(matmul(a, ones(64, 1))) != 128.0) return 3;

    // f16 is the same footprint; to_f64 widens back.
    let b = full(4, 64, 0.5);
    to_f16(b);
    if (tensor_nbytes(b) != 512) return 4;
    to_f64(b);
    if (tensor_nbytes(b) != 2048) return 5;

    // Model-level: cast every param at once, then widen back.
    let n = new Net();
    to_bf16(n);
    if (tensor_sum(n.forward(ones(64, 1))) != 128.0) return 6;
    to_f64(n);
    if (tensor_sum(n.forward(ones(64, 1))) != 128.0) return 7;
    return 0;
}
```

bf16 and f16 trade precision differently: for `0.1`, bf16 rounds to `0.100098` and f16 to `0.099976` — f16's extra mantissa bits make it finer, while bf16's wider exponent range makes it safer against overflow on large values. `0.5` (and any exact power-of-two combination) round-trips losslessly in both, which is why the checks above are exact.

Format	Cast	Storage	vs `f64`
`f64` (default)	`to_f64`	8 B/elem	1×
bf16	`to_bf16`	2 B/elem	4×
f16	`to_f16`	2 B/elem	4×

11.2 Loading a 16-bit checkpoint compact

A real checkpoint is often stored in bf16/f16 already. By default the loader **expands** such weights to `f64` on load (4× the memory). `load_compact(true)` tells it to keep them in their narrow form — `matmul` widens on the fly — so a bf16 model loads at a quarter of the peak memory. The on-disk format is standard safetensors, so files interoperate with other tools:

```
layer Src {
    param w: Tensor<f64, 64, 64> = full(64, 64, 0.5);
    forward(): Tensor<f64, 64, 64> { return this.w; }
}
layer Dst {
    param w: Tensor<f64, 64, 64> = load_safetensors("/tmp/abe_ml11.st", "w");
    forward(x: Tensor<f64, 64, 1>): Tensor<f64, 64, 1> { return matmul(this.w, x); }
}
layer Peek {
    param w: Tensor<f64, 64, 64> = load_safetensors("/tmp/abe_ml11.st", "w");
    forward(): Tensor<f64, 64, 64> { return this.w; }
}
function main(): i64 {
    let s = new Src();
    to_bf16(s);
    save_safetensors("/tmp/abe_ml11.st", "w", s.forward());   // standard BF16
safetensors
    load_compact(true);                                       // keep it compact on
reload
    let d = new Dst();
    if (tensor_sum(d.forward(ones(64, 1))) != 2048.0) return 1;
    let p = new Peek();
    if (tensor_nbytes(p.forward()) != 8192) return 2;         // compact bf16 (4096
* 2)
    return 0;
}
```

`load_compact` is **opt-in** because a compact weight is **matmul-only**: load the big projection weights compact and leave small `rmsnorm` / embedding weights on the default `f64` path (or widen them with `to_f64`).

11.3 bf16 training with `@amp`

Decorate a `train` statement with `@amp` to run a **bf16-numeric forward**: the forward's matmuls round their operands through bf16, while the master weights and the whole backward pass stay `f64` — the standard automatic-mixed-precision recipe. This reproduces the *numerics* of bf16 training so you can check that a model still converges under bf16 rounding:

```
ai {
    @amp(dtype: bf16)
    train m on dl {
        epochs: 2;
        optimizer: AdamW { lr: 1e-2; eps: 1e-8; weightDecay: 0.0; };
        on trainEnd(m) { console.log("amp train ok"); }
    }
}
```

`@amp` defaults to `bf16`; `@amp(dtype: fp16)` is rejected, because fp16 needs dynamic loss scaling that the `f64` backward can't meaningfully provide.

Tool	Purpose	
`to_bf16(t \	model) / to_f16(...)`	cast to 16-bit storage in place (4× smaller)
`to_f64(t \	model)`	widen back to `f64`
`load_compact(on)`	keep F16/BF16 weights compact on load (matmul-only)	
`@amp` on `train`	bf16-numeric forward, `f64` master weights + backward	

> **Scope.** bf16/f16 are **compact storage** for **inference** weights (matmul-only, like quantized tensors) — they are not yet trainable directly, and non-`matmul` kernels still need `f64`. `@amp` training reproduces bf16 *numerics* but keeps `f64` storage, so it is not yet a memory or throughput win — true narrow-storage training (with `f32` master weights) is a later step.

---

12. Running on the GPU

Everything so far runs on the CPU. Abe also has a **CUDA backend**: the same program, unchanged, can run its tensor ops on an NVIDIA GPU. Tensors stay **device-resident** (matmul uses cuBLAS; element-wise ops, softmax, and paged attention are CUDA kernels), so data isn't shuffled host ↔ device between ops.

Two steps — link it, then select it:

1. **Compile with `--gpu`** so the CUDA backend is linked in:

```
abec --gpu mymodel.abe -o mymodel
```

1. **Select the backend at run time** with `ABE_BACKEND`:

```
ABE_BACKEND=cuda ./mymodel # use the GPU (fall back to CPU + warn if none)
ABE_BACKEND=cpu ./mymodel # force CPU even if a GPU is present ./mymodel # auto: GPU if one is visible, else CPU
```

The program itself doesn't change. This one runs identically on either backend — the values are exact f64 fractions, so CPU and CUDA agree bit-for-bit:

```
function main(): i64 {
    let a = full(8, 16, 0.25);
    let b = full(16, 8, 0.5);
    let c = matmul(a, b);          // (8,8); each entry = 16 * 0.25 * 0.5 = 2.0
    let r = relu(c);               // 2.0 (unchanged, all positive)
    let s = tensor_sum(r);         // 8 * 8 * 2.0 = 128
    if (s == 128.0) { console.log("gpu parity ok"); }
    return 0;
}
```

Setting	Effect
`abec --gpu …`	link the CUDA backend (`libabe_gpu.a`); needs `nvcc` at build time

Setting	Effect
`ABE_BACKEND=cuda`	run on the GPU (warns + falls back to CPU if none is visible)
`ABE_BACKEND=cpu`	force the CPU backend
`ABE_BACKEND=auto` *(default)*	GPU if a CUDA device is visible, else CPU

> **Scope.** The GPU path is **f64** and built for **correctness** — it matches the CPU backend op-for-op (the test suite asserts CPU/CUDA parity), but the kernels are **not throughput-tuned** yet (no autotuning; whole-tensor softmax is single-block; transpose is 2-D only; non-last-axis softmax and RNG fall back to the host). Treat it as a correct reference GPU backend, not a peak-FLOPs one.

---

13. Gradients and einsum

Chapter 9 trained models with the `train` statement, which drives the autograd engine for you. Two expressions let you reach the same machinery directly.

13.1 `gradient(loss, params)`

`gradient(loss, params)` returns the reverse-mode derivative of a **scalar** `loss` with respect to a single tensor `params`. The loss is built from the param with ordinary tensor ops; `gradient` records them, runs the backward pass, and hands back the gradient (same shape as `params`):

```
function main(): i64 {
    let w: Tensor<f64, 1, 4> = full(1, 4, 0.0);
    let x: Tensor<f64, 4, 1> = full(4, 1, 3.0);
    let g = gradient(matmul(w, x), w);          // loss = w·x (1x1); d(loss)/dw = x
    if (tensor_sum(g) == 12.0) { console.log("gradient ok"); }   // 4 * 3
    console.log("grad numel:", tensor_numel(g));                 // 4
    return 0;
}
```

The loss must be a scalar (a size-1 tensor), exactly as in a training step.

13.2 `einsum`

`einsum("fmt", operands…)` contracts tensors by an index spec — the same notation NumPy/PyTorch use. The format is explicit (`inputs -> output`), labels are lowercase `a`–`z`, and up to 8 operands are supported:

```
function main(): i64 {
    let a = full(2, 3, 2.0);
    let b = full(3, 4, 0.5);
    let c = einsum("ij,jk->ik", a, b);   // matmul: (2,4), each = 3 * 2.0 * 0.5 =
3.0
    let t = einsum("ij->ji", a);         // transpose: (3,2)
    let s = einsum("ij->", a);           // sum-all: 12
    if (tensor_sum(c) == 24.0 && tensor_numel(t) == 6 && tensor_sum(s) == 12.0) {
        console.log("einsum ok");
```

```
    }
    return 0;
}
```

Handy patterns: `"ij,jk->ik"` (matmul), `"ij->ji"` (transpose), `"ii->i"` (diagonal), `"ij->"` (sum-all), `"bij,bjk->bik"` (batched matmul). Operands must be dense `f64` tensors — widen a compact bf16/quantized weight with `to_f64` first.

Expression	Purpose
`gradient(loss, params)`	reverse-mode d(loss)/d(params), scalar loss, one tensor param
`einsum("fmt", a, b, …)`	Einstein-summation contraction (1–8 dense f64 operands)
`jacobian(fn, x)`	reverse-mode Jacobian matrix of a top-level `fn: (Tensor)->Tensor`
`valueAndGrad(fn)(x)`	`[value, grad]` for a top-level scalar `fn`; `let (v, g) = …`

```
function f(x: Tensor<f64, 1, 4>): Tensor<f64, 1, 1> { return matmul(x, full(4, 1, 2.0)); }
function main(): i64 {
    let x: Tensor<f64, 1, 4> = full(1, 4, 1.0);
    let (v, g) = valueAndGrad(f)(x);   // v = sum(2x) = 8; g = [2,2,2,2]
    if (tensor_sum(v) == 8.0 && tensor_sum(g) == 8.0) { console.log("ok"); }
    return 0;
}
```

Scope. `gradient` differentiates a scalar loss w.r.t. one tensor. `jacobian(fn, x)` and `valueAndGrad(fn)(x)` take a top-level single-tensor-arg function (closures/arrows and partial application `valueAndGrad(fn)` aren't supported). `valueAndGrad` returns a 2-element list — destructure it with `let (v, g) = …` (positional `[a,b]` / `(a,b)` destructuring binding works generally now) or index `r[0]` / `r[1]`.

---

14. Convolution

For vision and signal models, `conv2d` and `conv1d` provide direct (f64) convolution — really cross-correlation, the ML convention — with zero padding and unit dilation:

- `conv2d(input, weight, stride, pad)` — `input` is `(N, Cin, H, W)` (or `(Cin, H, W)`), `weight` is `(Cout, Cin, KH, KW)`; output is `(N, Cout, Hout, Wout)` with `Hout = (H + 2·pad − KH)/stride + 1`.
- `conv1d(input, weight, stride, pad)` — the 1-D analogue: `(N, Cin, L)` with `(Cout, Cin, K)` → `(N, Cout, Lout)`.

```
function main(): i64 {
    // 3x3 input of ones, 2x2 kernel of ones, stride 1, pad 0 -> 2x2, each = 4.
```

```
    let x = full(1, 1, 3, 3, 1.0);
    let w = full(1, 1, 2, 2, 1.0);
    let y = conv2d(x, w, 1, 0);

    let xa = full(1, 1, 5, 1.0);
    let wa = full(1, 1, 3, 1.0);
    let ya = conv1d(xa, wa, 1, 0);       // length 3, each = 3

    if (tensor_sum(y) == 16.0 && tensor_numel(y) == 4 &&
        tensor_sum(ya) == 9.0 && tensor_numel(ya) == 3) {
        console.log("conv ok");
    }
    return 0;
}
```

Builtin	Shapes
`conv2d(input, weight, stride, pad)`	`(N,Cin,H,W)` ⊛ `(Cout,Cin,KH,KW)` → `(N,Cout,Hout,Wout)`
`conv1d(input, weight, stride, pad)`	`(N,Cin,L)` ⊛ `(Cout,Cin,K)` → `(N,Cout,Lout)`

> **Scope.** Direct f64 convolution, no bias term (add one with a broadcast), unit dilation, no groups. Fine for small CNNs and signal filters; not tuned for large-image throughput.

---

15. Object-oriented layers and models

A `layer` or `model` is not limited to a single `forward`. It can declare **custom methods**, a custom `**init()` constructor body, and (for models) a `config {}`** block of named hyperparameters — so a building block carries its own behaviour, not just its parameters.

15.1 Custom methods

A method other than `forward` lowers to `<Layer>__<method>(this, args…)` and dispatches the same way — callable from outside the instance and from `forward` via `this.<method>(…)`:

```
layer Block {
    param w: Tensor<f64, 4, 4> = full(4, 4, 0.5);
    project(x: Tensor<f64, 4, 1>): Tensor<f64, 4, 1> {   // a custom method
        return matmul(this.w, x);
    }
    forward(x: Tensor<f64, 4, 1>): Tensor<f64, 4, 1> {
        return this.project(x);            // call a custom method internally
    }
}
function main(): i64 {
    let b = new Block();
    let y = b.project(ones(4, 1));         // external dispatch: 4 * (4*0.5) = 8
    let f = b.forward(ones(4, 1));         // forward delegates to project: 8
```

```
    if (tensor_sum(y) == 8.0 && tensor_sum(f) == 8.0) { console.log("layer method
ok"); }
    return 0;
}
```

15.2 `init()` — a constructor body

Beyond `param = expr` field initializers, an `init()` block runs at construction **after** the params are allocated, with `this` in scope. It can read params, call methods, and assign params (`this.field = expr`):

```
layer Block {
    param w: Tensor<f64, 2, 2> = full(2, 2, 0.0);
    init() {
        this.w = full(2, 2, 3.0);          // compute & assign a param in init
    }
    forward(x: Tensor<f64, 2, 1>): Tensor<f64, 2, 1> { return matmul(this.w, x); }
}
function main(): i64 {
    let b = new Block();                    // init() runs here
    if (tensor_sum(b.forward(ones(2, 1))) == 12.0) { console.log("init ok"); }  //
2*(2*3)
    return 0;
}
```

15.3 Methods and `init()` on models

Models get the same surface — custom methods and an `init()` body, with internal `this.method(…)` dispatch:

```
model Net {
    shared param w: Tensor<f64, 4, 4> = full(4, 4, 0.0);
    init() { this.w = full(4, 4, 1.0); }                                    //
set in init
    score(x: Tensor<f64, 4, 1>): Tensor<f64, 4, 1> { return matmul(this.w, x); }  //
custom method
    forward(x: Tensor<f64, 4, 1>): Tensor<f64, 4, 1> { return this.score(x); }    //
internal dispatch
}
function main(): i64 {
    let n = new Net();
    // init set w=1.0 (not the 0.0 default); forward via the custom method:
4*(4*1.0) = 16
    if (tensor_sum(n.forward(ones(4, 1))) == 16.0) { console.log("model oo ok"); }
    return 0;
}
```

(`shared param` is covered in Chapter 16.)

15.4 `config {}` — model hyperparameters

A model `config {}` block declares named constants that are **in scope in the model's methods** (`forward`, custom methods, `init`), so the body reads them directly instead of threading them through every call:

```
model Hyper {
    config {
        layers: i64 = 12;
        scale:  f64 = 0.5;
    }
    forward(): i64 {
        return layers;          // a config value, in scope in the model body
    }
}
function main(): i64 {
    let h = new Hyper();
    if (h.forward() == 12) { console.log("config ok"); }
    return 0;
}
```

15.5 `noGrad` / `inferenceMode`

Inside an `ai {}` block, a `noGrad { … }` or `inferenceMode { … }` block disables gradient tracking for its body and restores the previous state on exit — the way to run pure inference without paying for the autograd tape:

```
layer Net {
    param w: Tensor<f64, 4, 4> = full(4, 4, 0.5);
    forward(x: Tensor<f64, 4, 1>): Tensor<f64, 4, 1> { return matmul(this.w, x); }
}
function main(): i64 {
    let n = new Net();
    let x: Tensor<f64, 4, 1> = ones(4, 1);
    ai {
        inferenceMode {                      // gradient tracking off inside the
block
            let y = n.forward(x);            // 4 rows * (4*0.5) = 8
            if (tensor_sum(y) == 8.0) { console.log("inference mode ok"); }
        }
        noGrad {                             // same, restored on exit
            let z = n.forward(x);
            if (tensor_sum(z) == 8.0) { console.log("no grad ok"); }
        }
    }
    return 0;
}
```

---

16. Weight tying

Tying weights — making two places in a network share one tensor — is standard in transformers (tied input embedding / output projection). Abe expresses it two ways.

16.1 `shared param` — one model-owned tensor

A model `shared param w: …` is allocated **once**, reachable as `this.w` everywhere in the model, and registered with the optimizer once. Using it in more than one place ties those uses to the same tensor:

```
model Tied {
    shared param w: Tensor<f64, 4, 4> = full(4, 4, 0.5);
    forward(x: Tensor<f64, 4, 1>): Tensor<f64, 4, 1> {
        return matmul(this.w, matmul(this.w, x));   // the same tied tensor, used
twice
    }
}
function main(): i64 {
    let m = new Tied();
    // w @ (w @ 1): inner row = 4*0.5 = 2; outer row = 4*0.5*2 = 4 -> sum = 16
    if (tensor_sum(m.forward(ones(4, 1))) == 16.0) { console.log("tie ok"); }
    return 0;
}
```

16.2 Cross-submodule tying

`this.<submodule>.<param>` reads and writes a submodule's parameter, so you can tie one submodule's weight to another — assign one to the other in `init()` and they share the same tensor afterwards:

```
layer Inner {
    param w: Tensor<f64, 2, 2> = full(2, 2, 0.5);
    forward(x: Tensor<f64, 2, 1>): Tensor<f64, 2, 1> { return matmul(this.w, x); }
}
model Net {
    a: Inner;
    b: Inner;
    init() {
        this.a.w = full(2, 2, 3.0);    // set a's weight (nested write)
        this.b.w = this.a.w;           // tie b to a (nested read + write -> shared
tensor)
    }
    forward(x: Tensor<f64, 2, 1>): Tensor<f64, 2, 1> { return this.b.forward(x); }
}
function main(): i64 {
    let n = new Net();
    // b.w is tied to a.w (=3): 2 rows * (2 * 3) = 12. Untied (default 0.5) would be
2.
    if (tensor_sum(n.forward(ones(2, 1))) == 12.0) { console.log("cross tie ok"); }
    return 0;
}
```

> **Training caveat.** A tied weight is registered once **per slot**, so a shared weight that is trained may have its update counted more than once. This is harmless for inference; for training, prefer the single `shared param` form (16.1), which registers exactly once.

---

17. Mixture of Experts

Abe has the primitives for a Mixture-of-Experts (MoE) layer: a learned router gate, top-k gating math, and the Switch-Transformer load-balancing loss. The expert **dispatch/combine** itself is ordinary `forward` code (loop the expert submodules, weight each by its gate column) — it can't be a runtime builtin, because that would have to call back into your expert layers.

17.1 A `router` gate

`router name: Tensor<…> = …;` declares a learned gating weight inside a layer. It is a trainable param-shaped slot: allocated in the constructor, reachable as `this.<name>`, and registered with the optimizer. Experts are ordinary submodules alongside it:

```
layer Expert {
    param w: Tensor<f64, 4, 4> = full(4, 4, 0.5);
    forward(x: Tensor<f64, 4, 1>): Tensor<f64, 4, 1> { return matmul(this.w, x); }
}
layer MoE {
    router gate: Tensor<f64, 2, 4> = full(2, 4, 0.25);   // learned gate over D=4
for 2 experts
    param e0: Expert;
    param e1: Expert;
    forward(x: Tensor<f64, 4, 1>): Tensor<f64, 2, 1> {
        let _y0 = this.e0.forward(x);          // experts coexist with the router
        let _y1 = this.e1.forward(x);
        return softmax(matmul(this.gate, x));  // gate logits -> gate weights
    }
}
function main(): i64 {
    let m = new MoE();
    // gate=0.25, x=ones(4,1) -> logits [1,1] -> softmax [0.5,0.5] -> sum 1.0
    if (tensor_sum(m.forward(ones(4, 1))) == 1.0) { console.log("moe router ok"); }
    return 0;
}
```

17.2 Gating builtins: `xavier_uniform`, `topk_gate`

`xavier_uniform(rows, cols)` is a Glorot-uniform initializer (the standard init for a router gate), shaped like `randn`. `topk_gate(scores, k)` keeps the `k` largest scores per row (over the last axis = experts), zeros the rest, and renormalizes the kept weights to sum to 1 — the top-k gating math:

```
function main(): i64 {
    // Glorot init: (3,4) -> 12 values in [-sqrt(6/7), sqrt(6/7)].
    let gate = xavier_uniform(3, 4);
    // Top-k gating over 4 experts (last axis): keep top-2, renormalize.
    let scores = full(1, 4, 1.0);
    let g = topk_gate(scores, 2);              // two equal winners -> 0.5, 0.5
    if (tensor_numel(gate) == 12 &&
        tensor_sum(g) == 1.0 && tensor_numel(g) == 4) {
        console.log("moe dispatch ok");
    }
```

```
    return 0;
}
```

17.3 Load-balancing auxiliary loss

`moe_balance_loss(gate_probs, k)` returns the scalar $E \cdot \Sigma_i f_i \cdot P_i$ (Switch Transformer), where P_i is the mean router probability for expert `i` and f_i is the fraction of tokens that route expert `i` into their top-k. It is minimized by balanced routing; add it (times a small coefficient) to the task loss to discourage expert collapse. It is differentiable through P_i, so backprop nudges the gate toward balance:

```
function main(): i64 {
    // 1 token, 4 experts, uniform probs; top-2 -> experts 0,1 (tie-break).
    // P=[.25,.25,.25,.25], f=[1,1,0,0]  ->  aux = 4*(.25+.25) = 2.0
    let probs = full(1, 4, 0.25);
    let aux = moe_balance_loss(probs, 2);
    // gradient flows through P_i: d(aux)/d(probs[0,i]) = (E/N)*f_i = 4*f_i ->
[4,4,0,0], sum 8
    let p2 = full(1, 4, 0.25);
    let g = gradient(moe_balance_loss(p2, 2), p2);
    if (tensor_sum(aux) == 2.0 && tensor_sum(g) == 8.0) { console.log("moe balance
ok"); }
    return 0;
}
```

Builtin	Purpose
`router name: T = …`	a learned, trainable gating weight in a layer
`xavier_uniform(rows, cols)`	Glorot-uniform initializer (router-gate init)
`topk_gate(scores, k)`	top-k renormalized gating weights over the last axis
`moe_balance_loss(probs, k)`	Switch-Transformer load-balancing aux loss (differentiable)

---

18. Custom gradients

A layer that declares a `backward(gradOut): gradIn` method gets a **custom gradient** (PyTorch `Function.backward` style): its `forward` runs untaped and the autograd engine calls `backward` to turn the output gradient into the input gradient. The clean use is a custom-gradient op or activation — it is param-less (parameter gradients inside such a layer are not auto-computed).

The example below deliberately makes `backward` return `3·gradOut` while `forward` computes `2·x`, so the result (3, not 2) proves the custom backward is the path actually taken:

```
layer Op {
    forward(x: Tensor<f64, 1, 1>): Tensor<f64, 1, 1> { return x + x; }        // 2x
    backward(g: Tensor<f64, 1, 1>): Tensor<f64, 1, 1> { return g + g + g; }   //
custom: 3*g
}
function main(): i64 {
```

```
    let op = new Op();
    let x: Tensor<f64, 1, 1> = full(1, 1, 5.0);
    let grad = gradient(op.forward(x), x);     // scalar loss; custom backward -> 3
    if (tensor_sum(grad) == 3.0) { console.log("custom backward ok"); }
    return 0;
}
```

---

19. Knowledge distillation

`distill student on <data> from teacher { … }` trains a small **student** to imitate a larger **teacher**. Each step runs the frozen teacher forward (untaped), the student forward (taped), and a KD loss that blends a soft teacher-matching term (at `temperature` T) with the hard cross-entropy (weight `1 − alpha`); only the student is optimized. The block takes the same `epochs` / `optimizer` / event hooks as `train` (Chapter 9), plus `temperature` and `alpha`:

```
extern function abe_dataset_new(): ptr;
extern function abe_dataset_demo_tokens(ds: ptr, mod_v: i64, count: i64): void;
extern function abe_dataloader_new(ds: ptr, b: i64, s: i64, shuf: bool, seed: i64):
ptr;

layer Embed<V, D> {
    param table: Tensor<f64, V, D> = randn(V, D);
    forward(ids: Tensor<f64>): Tensor<f64> { return embedding_lookup(this.table,
ids); }
}
layer Proj<D, V> {
    param weight: Tensor<f64, D, V> = randn(D, V);
    forward(x: Tensor<f64>): Tensor<f64> { return matmul(x, this.weight); }
}
model Tiny<V, D> {
    embed: Embed<V, D>;
    proj:  Proj<D, V>;
    forward(ids: Tensor<f64>): Tensor<f64> { return
this.proj.forward(this.embed.forward(ids)); }
}
function main(): i64 {
    let student = new Tiny<8, 4>();
    let teacher = new Tiny<8, 4>();
    const ds = abe_dataset_new();
    abe_dataset_demo_tokens(ds, 8, 64);
    const dl = abe_dataloader_new(ds, 2, 4, false, 0);
    ai {
        distill student on dl from teacher {
            epochs: 2;
            temperature: 2.0;
            alpha: 0.5;
            optimizer: AdamW { lr: 1e-2; eps: 1e-8; weightDecay: 0.0; };
            on trainEnd(m) { console.log("distill ok"); }
```

```
        }
    }
    return 0;
}
```

The soft KD loss decreases as the student approaches the teacher (measured separately at roughly 0.041 → 0.001 over a few epochs on this fixture).

---

20. Conversational AI

Abe has statement forms for assembling prompts and running a simple interactive dialog. `prompt { … }` (inside `ai {}`) joins its property values into one newline-separated string; the `dialog {}` block adds `ask` / `respond` / `confirm`:

- `prompt { system; user; … }` → one newline-joined prompt string.
- `ask user { prompt: … }` → ask the user and bind the reply text.
- `respond { message: … }` → print a reply.
- `confirm { prompt: … }` → ask a yes/no question and bind a `bool`.

```
function main(): i64 {
    ai {
        // prompt { … } joins its property values into one newline-joined string.
        let p = prompt { system: "SYS"; user: "USR"; };
        console.log("prompt:", p);          // SYS\nUSR
    }
    dialog {
        let name = ask user { prompt: "Name?"; };   // ask + bind the reply
        respond { message: name; };                  // print a reply
        let ok = confirm { prompt: "Continue?"; };   // yes/no -> bool
    }
    return 0;
}
```

`ask` / `confirm` read a line from standard input; on end-of-input they yield an empty reply / the default, so a non-interactive run does not block.

---

21. Distributed training

`@ddp(worldSize: N)` on a `train` runs a **single-process** simulation of Distributed Data Parallel: the optimizer accumulates and averages gradients over `N` consecutive micro-batches, then takes one real step — the same math DDP performs across `N` GPUs, giving an `N`× larger effective batch. It is **not** true multi-GPU/NCCL on this build, and the compiler says so with an info diagnostic rather than pretending otherwise:

```
extern function abe_dataset_new(): ptr;
extern function abe_dataset_demo_tokens(ds: ptr, mod_v: i64, count: i64): void;
```

```
extern function abe_dataloader_new(ds: ptr, b: i64, s: i64, shuf: bool, seed: i64):
ptr;

layer Embed<V, D> {
    param table: Tensor<f64, V, D> = randn(V, D);
    forward(ids: Tensor<f64>): Tensor<f64> { return embedding_lookup(this.table,
ids); }
}
layer Proj<D, V> {
    param weight: Tensor<f64, D, V> = randn(D, V);
    forward(x: Tensor<f64>): Tensor<f64> { return matmul(x, this.weight); }
}
model Tiny<V, D> {
    embed: Embed<V, D>;
    proj:  Proj<D, V>;
    forward(ids: Tensor<f64>): Tensor<f64> { return
this.proj.forward(this.embed.forward(ids)); }
}
function main(): i64 {
    let m = new Tiny<8, 4>();
    const ds = abe_dataset_new();
    abe_dataset_demo_tokens(ds, 8, 64);
    const dl = abe_dataloader_new(ds, 2, 4, false, 0);
    ai {
        @ddp(worldSize: 4)
        train m on dl {
            epochs: 2;
            optimizer: AdamW { lr: 1e-2; eps: 1e-8; weightDecay: 0.0; };
            on trainEnd(m) { console.log("ddp train ok"); }
        }
    }
    return 0;
}
```

`@fsdp` / `@tensorParallel` / `@pipeline` are accepted and acknowledged (with an info diagnostic) but have no effect on this single-process build — real parameter/tensor/pipeline sharding needs multi-GPU hardware and NCCL collectives.

---

22. ONNX interoperability

Abe can load and run a pre-trained **ONNX** model through onnxruntime:

- `onnx_load(path)` → an opaque session handle (`null` on failure).
- `onnx_run(session, x)` → the output `Tensor`.
- `onnx_free(session)` → release the session.

onnxruntime is an **optional** dependency: the runtime is built against it by pointing `ORT_HOME` at an onnxruntime install (see `runtime/Makefile` and `make -C runtime test-onnx`). Without it,

`onnx_load` returns `null` and prints an honest "not available" message — so guard the handle and the same source runs either way:

```
function main(): i64 {
    let m = onnx_load("/tmp/relu_f64.onnx");
    if (m == null) {                       // runtime built without onnxruntime
        console.log("onnx ok (stub: onnxruntime not built)");
        return 0;
    }
    let y = onnx_run(m, full(4, 1, 3.0));  // Relu([3,3,3,3]) = [3,3,3,3], sum 12
    if (tensor_sum(y) == 12.0) { console.log("onnx ok"); }
    onnx_free(m);
    return 0;
}
```

The feed/fetch tensors are `f64`; the model's input/output element type must be `double` (the integration's end-to-end test uses a hand-encoded `Relu` double model, verified to produce `[0,2,0,4]` from `[-1,2,-3,4]`).

Appendix — Current limitations

Abe's ML stack runs a complete Llama-class inference and training pipeline today, but it is still maturing. So you know exactly what to expect, this release does **not** yet include:

- **Precision / hardware:** tensor math accumulates in `f64` on both backends. The **CUDA GPU backend works and matches the CPU op-for-op** (Chapter 12), but it is a correct reference — not throughput-tuned (no autotuning; a few ops fall back to the host). Mixed precision is available as **compact 16-bit storage** (bf16/f16; Chapter 11) and a **bf16-numeric `@amp` forward** for training — but storage-side bf16/f16 is inference-only (matmul widens to `f64`), and fp16 with loss scaling is not wired.
- **Quantization:** Q8_0 (8-bit) and Q4_0 (4-bit) **weight** quantization are available (Chapter 10) — quantize, run, and save/load compact. Not yet: loading off-the-shelf **GGUF** models, 4-bit super-block formats (Q4_K), and integer (int8) matmul for speed — quantization shrinks memory, not yet runtime.
- **Distributed training:** `@ddp(worldSize: N)` on a `train` runs a single-process simulation — gradients are accumulated and averaged over N micro-batches per optimizer step (the DDP math, = an N× effective batch), not true multi-GPU. `@fsdp` / `@tensorParallel` / `@pipeline` are acknowledged (with an info diagnostic) but have no effect: real cross-device sharding + NCCL collectives need multi-GPU hardware.
- **Autograd reach:** explicit autograd (`gradient`, `einsum`, `jacobian`, `valueAndGrad`; Chapter 13) covers a scalar loss / a top-level single-tensor-arg function. Partial application `valueAndGrad(fn)` and closure/arrow `fn`s aren't supported — they need first-class function values. A layer custom `backward` (Chapter 18) is a param-less custom-gradient op: parameter gradients inside such a layer are not auto-computed.
- **MoE dispatch:** the router gate, top-k gating, and load-balancing loss are builtins (Chapter 17), but the expert **dispatch/combine** is ordinary `forward` code — it can't be a builtin, which would have to call your expert layers.

- **Weight tying:** a tied weight registered **per slot** (the cross-submodule form, 16.2) may double-count its update during training; prefer a single `shared param` (16.1) when training a tied weight. (Inference is unaffected.)
- **ONNX:** onnxruntime is an **optional** dependency (Chapter 22). A default build links an honest "not available" stub (`onnx_load` returns `null`); real inference needs a runtime built against onnxruntime (`ORT_HOME`). Feed/fetch tensors are `f64` (the model's I/O element type must be `double`).
- **Other ops:** `conv2d`/`conv1d` are direct f64 (Chapter 14; no groups/bias/dilation); `reshape` always copies. `matmul` operates on 2-D tensors, which is why layer inputs use the column form `Tensor<f64, In, 1>`.

These constraints are the roadmap, not the destination.

Abe Pro ML Manual (Level 2) — complete: Chapters 1–22 (Tensors, Layers, Models, Attention, Inference, Tokenization, Serving, Loading models, Training, Quantization, Mixed precision, Running on the GPU, Gradients and einsum, Convolution, Object-oriented layers and models, Weight tying, Mixture of Experts, Custom gradients, Knowledge distillation, Conversational AI, Distributed training, ONNX interoperability). Every example is compiled and run by `tests/run_manual_ml_examples.sh` (`make test-manual-ml`).

APPENDIX

Runtime Function Reference

The Abe Pro runtime exposes **523** functions that the compiler links into native programs. In day-to-day code you reach these through language constructs (method calls, operators, built-ins) documented in Parts I–II; this index is a completeness reference to the underlying runtime surface, grouped by level.

Level 1 — general runtime

Core / runtime (61)

```
abe_core_all_ok  abe_core_all_some  abe_core_alloc  abe_core_assert  abe_core_ensure
abe_core_err  abe_core_first_ok  abe_core_first_some  abe_core_free  abe_core_guard
abe_core_nothing  abe_core_ok  abe_core_option_and  abe_core_option_expect
abe_core_option_filter  abe_core_option_filter_with_ctx  abe_core_option_flat_map
abe_core_option_flat_map_with_ctx  abe_core_option_flatten  abe_core_option_is_none
abe_core_option_is_some  abe_core_option_map  abe_core_option_map_or
abe_core_option_map_or_else  abe_core_option_map_with_ctx  abe_core_option_ok_or
abe_core_option_ok_or_else  abe_core_option_or  abe_core_option_or_else
abe_core_option_replace  abe_core_option_take  abe_core_option_unwrap
abe_core_option_unwrap_or  abe_core_option_xor  abe_core_option_zip
abe_core_option_zip_with  abe_core_panic  abe_core_require  abe_core_result_and
abe_core_result_err  abe_core_result_expect  abe_core_result_expect_err
abe_core_result_flat_map  abe_core_result_flat_map_with_ctx  abe_core_result_flatten
abe_core_result_is_err  abe_core_result_is_ok  abe_core_result_map
abe_core_result_map_err  abe_core_result_map_err_with_ctx  abe_core_result_map_or
abe_core_result_map_or_else  abe_core_result_map_with_ctx  abe_core_result_ok
abe_core_result_or  abe_core_result_or_else  abe_core_result_unwrap
abe_core_result_unwrap_err  abe_core_result_unwrap_or  abe_core_some
abe_core_zalloc
```

Strings (20)

```
abe_string_char_at  abe_string_compare  abe_string_concat  abe_string_equals
abe_string_free  abe_string_index_of  abe_string_length  abe_string_match_regex
abe_string_matches  abe_string_new  abe_string_pad_end  abe_string_pad_start
abe_string_replace_all_regex  abe_string_replace_regex  abe_string_split
abe_string_split_regex  abe_string_substring  abe_string_to_lower
abe_string_to_upper  abe_string_trim
```

Lists (27)

```
abe_list_capacity  abe_list_clear  abe_list_clone  abe_list_concat
abe_list_contains  abe_list_filter  abe_list_for_each  abe_list_free  abe_list_get
abe_list_index_of  abe_list_insert  abe_list_is_empty  abe_list_iter_free
abe_list_iter_has_next  abe_list_iter_new  abe_list_iter_next  abe_list_length
abe_list_map  abe_list_new  abe_list_new_capacity  abe_list_pop  abe_list_push
```

```
abe_list_release_boxed_scalars  abe_list_remove  abe_list_reverse  abe_list_set
abe_list_slice
```

Maps (22)

```
abe_map_clear  abe_map_clone  abe_map_delete  abe_map_entries  abe_map_free
abe_map_get  abe_map_get_str  abe_map_has  abe_map_has_str  abe_map_is_empty
abe_map_iter_free  abe_map_iter_has_next  abe_map_iter_new  abe_map_iter_next_key
abe_map_iter_next_value  abe_map_keys  abe_map_new  abe_map_new_capacity
abe_map_set  abe_map_set_str  abe_map_size  abe_map_values
```

Sets (22)

```
abe_set_add  abe_set_clear  abe_set_clone  abe_set_delete  abe_set_difference
abe_set_free  abe_set_has  abe_set_intersection  abe_set_interval  abe_set_is_empty
abe_set_is_subset  abe_set_is_superset  abe_set_iter_free  abe_set_iter_has_next
abe_set_iter_new  abe_set_iter_next  abe_set_new  abe_set_new_capacity  abe_set_size
abe_set_timeout  abe_set_to_list  abe_set_union
```

Typed arrays (9)

```
abe_array_alloc  abe_array_copy  abe_array_equals  abe_array_fill  abe_array_find
abe_array_free  abe_array_sort_f64  abe_array_sort_i64  abe_array_sort_str
```

Dates (17)

```
abe_date_free  abe_date_get_day  abe_date_get_day_of_week  abe_date_get_hours
abe_date_get_milliseconds  abe_date_get_minutes  abe_date_get_month
abe_date_get_seconds  abe_date_get_time  abe_date_get_year  abe_date_micros
abe_date_new  abe_date_now  abe_date_now_obj  abe_date_to_iso_string
abe_date_to_string  abe_date_unix
```

Regex (16)

```
abe_regex_compile  abe_regex_exec  abe_regex_free  abe_regex_match
abe_regex_match_all  abe_regex_match_free  abe_regex_match_groups
abe_regex_match_index  abe_regex_match_length  abe_regex_match_string  abe_regex_new
abe_regex_pattern  abe_regex_replace  abe_regex_replace_all  abe_regex_split
abe_regex_test
```

I/O & files (13)

```
abe_io_input  abe_io_open  abe_io_print  abe_io_print_f64  abe_io_print_i64
abe_io_println  abe_io_println_err  abe_io_println_err_bool  abe_io_println_err_f64
abe_io_println_err_i64  abe_io_println_i64  abe_io_read_file  abe_io_write_file
```

HTTP client/server (23)

```
abe_http_cleanup  abe_http_delete  abe_http_fetch  abe_http_get  abe_http_head
abe_http_options_free  abe_http_options_new  abe_http_options_set_body
abe_http_options_set_header  abe_http_options_set_method
abe_http_options_set_timeout  abe_http_patch  abe_http_post  abe_http_put
abe_http_response_body  abe_http_response_free  abe_http_response_headers
abe_http_response_new  abe_http_response_ok  abe_http_response_status
abe_http_response_text  abe_http_url_decode  abe_http_url_encode
```

Crypto & auth (10)

```
abe_auth_cleanup_sessions  abe_auth_create_session  abe_auth_csrf_token
abe_auth_destroy_all_sessions  abe_auth_destroy_session  abe_auth_generate_token
abe_auth_hash_password  abe_auth_session_token  abe_auth_validate_session
abe_auth_verify_password
```

Promises (16)

```
abe_promise_all  abe_promise_await  abe_promise_create  abe_promise_free
abe_promise_get_error  abe_promise_get_result  abe_promise_get_state
abe_promise_is_pending  abe_promise_is_rejected  abe_promise_is_resolved
abe_promise_race  abe_promise_reject  abe_promise_rejected  abe_promise_resolve
abe_promise_resolved  abe_promise_then
```

Async (6)

```
abe_async_alloc  abe_async_delay  abe_async_free  abe_async_run  abe_async_schedule
abe_async_sleep_ms
```

Task groups (8)

```
abe_taskgroup_cancel  abe_taskgroup_cancelled  abe_taskgroup_fail
abe_taskgroup_free  abe_taskgroup_new  abe_taskgroup_spawn
abe_taskgroup_spawn_closure  abe_taskgroup_wait
```

Actors (8)

```
abe_actor_call  abe_actor_call_closure  abe_actor_free  abe_actor_new
abe_actor_send  abe_actor_send_closure  abe_actor_state  abe_actor_stop
```

Runtime support (3)

```
abe_rt_cross_entropy  abe_rt_kd_loss  abe_rt_moe_balance_loss
```

Level 2 — machine learning runtime

Tensors (5)

```
abe_tensor_quantize_q4  abe_tensor_quantize_q8  abe_tensor_to_bf16
abe_tensor_to_f16  abe_tensor_to_f64
```

ML ops & layers (68)

```
abe_ml_arange  abe_ml_conv1d  abe_ml_conv2d  abe_ml_cross_entropy  abe_ml_einsum
abe_ml_embedding_lookup  abe_ml_eye  abe_ml_full  abe_ml_gqa_attention
abe_ml_kd_loss  abe_ml_linspace  abe_ml_load_model  abe_ml_model_free
abe_ml_model_predict  abe_ml_moe_balance_loss  abe_ml_ones  abe_ml_onnx_free
abe_ml_onnx_input_names  abe_ml_onnx_input_shape  abe_ml_onnx_load
abe_ml_onnx_load_with_provider  abe_ml_onnx_num_inputs  abe_ml_onnx_num_outputs
abe_ml_onnx_output_names  abe_ml_onnx_output_shape  abe_ml_onnx_run
abe_ml_onnx_run_multi  abe_ml_random  abe_ml_relu  abe_ml_reshape  abe_ml_rmsnorm
abe_ml_rope  abe_ml_sigmoid  abe_ml_silu  abe_ml_softmax  abe_ml_swiglu
abe_ml_tanh_tensor  abe_ml_tensor_add  abe_ml_tensor_clone  abe_ml_tensor_create
abe_ml_tensor_data  abe_ml_tensor_div  abe_ml_tensor_free  abe_ml_tensor_get
abe_ml_tensor_matmul  abe_ml_tensor_max  abe_ml_tensor_mean  abe_ml_tensor_min
abe_ml_tensor_mul  abe_ml_tensor_nbytes  abe_ml_tensor_ndim  abe_ml_tensor_neg
abe_ml_tensor_new_ggml  abe_ml_tensor_new_quant  abe_ml_tensor_print
abe_ml_tensor_reshape  abe_ml_tensor_scale  abe_ml_tensor_set
abe_ml_tensor_set_flat  abe_ml_tensor_shape  abe_ml_tensor_size  abe_ml_tensor_std
abe_ml_tensor_sub  abe_ml_tensor_sum  abe_ml_tensor_transpose  abe_ml_topk_gate
abe_ml_xavier_uniform  abe_ml_zeros
```

Autograd (31)

```
abe_autograd_backward  abe_autograd_checkpoint  abe_autograd_custom_backward
abe_autograd_disable  abe_autograd_enable  abe_autograd_grad
abe_autograd_is_enabled  abe_autograd_jacobian  abe_autograd_record_add
abe_autograd_record_concat  abe_autograd_record_cross_entropy
abe_autograd_record_embedding  abe_autograd_record_gqa  abe_autograd_record_kd_loss
abe_autograd_record_matmul  abe_autograd_record_moe_balance
abe_autograd_record_relu  abe_autograd_record_reshape  abe_autograd_record_rmsnorm
abe_autograd_record_sigmoid  abe_autograd_record_silu  abe_autograd_record_slice
abe_autograd_record_softmax  abe_autograd_record_softmax_axis
abe_autograd_record_swiglu  abe_autograd_record_transpose
abe_autograd_register_param  abe_autograd_set_enabled  abe_autograd_tape_clear
abe_autograd_value_and_grad  abe_autograd_zero_grad
```

Optimizers (11)

```
abe_optim_adamw_new  abe_optim_add_param  abe_optim_clip_grad_norm  abe_optim_free
abe_optim_get_lr  abe_optim_set_accum  abe_optim_set_lr  abe_optim_sgd_new
abe_optim_step  abe_optim_step_count  abe_optim_zero_grad
```

Mixed precision (6)

```
abe_amp_is_active  abe_amp_loss_scale  abe_amp_matmul  abe_amp_scale_loss
abe_amp_set_active  abe_amp_unscale_grads
```

Model save/load (3)

```
abe_saver_add  abe_saver_finish  abe_saver_new
```

GGUF interop (6)

```
abe_gguf_block_bytes  abe_gguf_decode  abe_gguf_global_close  abe_gguf_global_load
abe_gguf_global_open  abe_gguf_save_one
```

Conversational / prompt (33)

```
abe_prompt_anthropic  abe_prompt_chat  abe_prompt_complete  abe_prompt_custom
abe_prompt_message_assistant  abe_prompt_message_content  abe_prompt_message_free
abe_prompt_message_new  abe_prompt_message_role  abe_prompt_message_system
abe_prompt_message_user  abe_prompt_openai  abe_prompt_options_free
abe_prompt_options_new  abe_prompt_options_set_max_tokens
abe_prompt_options_set_stop  abe_prompt_options_set_temperature
abe_prompt_options_set_top_p  abe_prompt_provider_free  abe_prompt_provider_new
abe_prompt_provider_set_model  abe_prompt_provider_set_org
abe_prompt_provider_set_timeout  abe_prompt_response_completion_tokens
abe_prompt_response_content  abe_prompt_response_finish_reason
abe_prompt_response_free  abe_prompt_response_new  abe_prompt_response_prompt_tokens
abe_prompt_response_role  abe_prompt_response_total_tokens  abe_prompt_simple
abe_prompt_with_system
```

Dialog (11)

```
abe_dialog_ask  abe_dialog_ask_with_default  abe_dialog_ask_with_options
abe_dialog_confirm  abe_dialog_confirm_with_default  abe_dialog_respond
abe_dialog_respond_formatted  abe_dialog_turn_confirmed  abe_dialog_turn_free
abe_dialog_turn_text  abe_dialog_turn_timestamp
```

Abe Pro Programming Manual, Version 1.0.0. See the Abe Pro Administration Manual for installation, runtime distribution, and license administration.